



نسخه 1.2.0 · راهنمای کامل کاربر

Tehran Stock Exchange Data & Analytics Library

کتابخانه داده و تحلیل بازار سرمایه ایران

Stocks · Live Market · Options · ETFs · Bonds · Currency

سهام · بازار زنده · اختیار معامله · صندوق · اوراق بدهی · ارز و سکه

Python 3.8–3.14

1 September 2026

algotik.com

Mohsen Alipour · @algotik

راهنمای مطالعه

چطور از این دفترچه استفاده کنید؟

این راهنما دو لایه دارد: ابتدا مسیرهای سریع و توابعی که بیشتر کاربران هر روز نیاز دارند؛ سپس مرجع کامل ورودی‌ها، گزینه‌ها، خروجی‌ها و نکات رفتاری همهٔ API‌های عمومی پکیج.

۱

شروع سریع

نصب، import و اولین دریافت تاریخچهٔ قیمت در چند دقیقه.

۲

توابع پرکاربرد

قیمت، حقیقی/حقوقی، بازار زنده، سفارش، ارز، صندوق، اخزا و اختیار.

۳

قراردادهای داده

نوع خروجی، تاریخ، freshness، ردیف امروز، CSV و مرز منابع داده.

۴

مرجع تفصیلی

signatureها، تمام ورودی‌ها، مقادیرهای مجاز، پیش‌فرض‌ها و خروجی هر خانوادهٔ تابع.

نکتهٔ معاملاتی: پیش از استفاده در تصمیم‌گیری یا اجرای الگوریتم، زمان snapshot، ویژگی‌های `DataFrame.attrs`، وضعیت `partial` و freshness داده را بررسی کنید. این پکیج توصیهٔ سرمایه‌گذاری ارائه نمی‌کند.

A Python toolkit for historical, live and analytical data from Iran's capital market.

Fetch TSETMC prices, client type, trades, order books, funds, bonds and options with Jalali date support, then use the built-in fixed-income and option analytics for research and algorithmic trading.

IR معرفی فارسی

برای دریافت و تحلیل داده‌های بورس و فرابورس ایران ساخته شده است. با نام فارسی نماد می‌توانید تاریخچه `algotik-tse` قیمت، حقیقی/حقوقی، معاملات ریز، سفارش‌ها و اطلاعات لحظه‌ای بازار را بگیرید؛ شاخص‌های صنعت و اعضای دقیق آن‌ها را تحلیل کنید؛ فهرست صندوق‌ها، اوراق و اختیارها را بسازید؛ و تحلیل‌های تخصصی اخزا و اختیار معامله را روی همان داده‌ها انجام دهید.

بیشتر خروجی‌های جدولی به صورت **Pandas DataFrame** ارائه می‌شوند و تاریخ شمسی، تاریخ میلادی و داده‌های چندنمادی پشتیبانی می‌شوند. خروجی‌های ساختاریافته‌ای مانند snapshot بازار، زنجیره اختیار و منحنی بازده در بخش **نوع خروجی** توضیح داده شده‌اند.

ویژگی‌ها

- دریافت تاریخچه قیمت و حقیقی/حقوقی با نماد فارسی، تاریخ شمسی/میلادی، تعدیل قیمت، بازده و خروجی چندنمادی
- افزودن کنترل‌شده ردیف امروز با `include_today=True` و metadata مربوط به freshness و partial بودن داده
- نمای زنده کل بازار یا یک نماد، قدرت خریدار، جریان پول، spread و imbalance پنج سطح سفارش
- معاملات ریز، order book و صف خرید/فروش به صورت زنده و تاریخی
- SQLite جریان صنایع و ذخیره اختیاری تاریخچه روی breadth، بازار، پیام‌ها، تغییر وضعیت watcher
- ۴۵ شاخص صنعت، اعضای رسمی، snapshot تحلیلی، تاریخچه اعضا، کندل درون‌روزی و رتبه‌بندی صنایع
- `InsCode` صندوق‌های قابل معامله، رخدادهای تعدیل قیمت و حل دقیق هویت ابزار با P/E و EPS
- تحلیل اخزا شامل DV01، duration، convexity، YTM و منحنی بازده زنده و تاریخی
- تحلیل اختیار معامله شامل PCR، put-call parity، Greeks، IV، Black-Scholes و نقدشوندگی
- ارز و سکه، اینترادی، سهامداران، شاخص‌ها، ETFها، صندوق‌ها و اوراق بدهی
- و محدودسازی منبع داده retry، rate limiting، به صورت پیش‌فرض TLS و اعتبارسنجی HTTPS

وبسایت: algotik.com | تلگرام: t.me/algotik

این کتابخانه توصیه سرمایه‌گذاری نیست. برای استفاده معاملاتی، زمان snapshot، freshness، partial بودن داده و `DataFrame.attrs` را بررسی کنید.

API در یک نگاه

اگر اولین بار است از پکیج استفاده می‌کنید، معمولاً همین توابع نیاز شما را پوشش می‌دهند:

تابع	چه کاری انجام می‌دهد؟	خروجی
★ <code>get_history()</code>	تاریخچه OHLCV یک یا چند نماد	<code>DataFrame</code>
★ <code>get_client_type()</code>	تاریخچه خریدوفروش حقیقی/حقوقی	<code>DataFrame</code>
★ <code>get_live_symbol()</code>	نمای زنده و تحلیلی یک نماد	یک ردیفی <code>DataFrame</code>
★ <code>get_live_market()</code>	نمای زنده و تحلیلی کل بازار	<code>DataFrame</code>
★ <code>get_market_snapshot()</code>	خام قیمت و پنج سطح سفارش کل بازار snapshot	<code>dict</code>
★ <code>get_intraday()</code>	تیک یا کندل اینترادی	قدیمی API در <code>None</code> یا <code>DataFrame</code>
★ <code>get_symbols()</code>	فهرست نمادها بر اساس بازار و نوع ابزار	یا فهرست <code>DataFrame</code>
★ <code>get_currency()</code>	تاریخچه ارز و سکه	<code>DataFrame</code>
★ <code>get_live_trades()</code>	معاملات ریز امروز یک نماد	<code>DataFrame</code>
★ <code>get_order_book()</code>	پنج سطح سفارش زنده	<code>DataFrame</code>
★ <code>get_industry_snapshot()</code>	بازده، breadth، جریان پول و صف یک یا همه صنایع	<code>DataFrame</code>
★ <code>rank_industries()</code>	رتبه‌بندی صنایع با معیار انتخابی	<code>DataFrame</code>
★ <code>list_etfs()</code>	NAV ها همراه قیمت و ETF	<code>DataFrame</code>
★ <code>list_funds()</code>	صندوق‌ها همراه NAV، بازده و ترکیب دارایی	<code>DataFrame</code>

نقشه کامل توابع کاربری

جدول‌های زیر API های canonical را نشان می‌دهند. نام‌های قدیمی مانند `stock()` و `stock_RI()` همچنان کار می‌کنند، اما برای کد جدید نام‌های `get_*` پیشنهاد می‌شوند.

قیمت، حقیقی/حقوقی و اطلاعات نماد

تابع	کاربرد
<code>get_history()</code>	تاریخچه قیمت تعدیل‌شده/خام، بازده و داده چندنمادی
<code>get_client_type()</code>	تاریخچه حقیقی/حقوقی و قدرت خریدار
<code>get_intraday()</code>	تیک و کندل‌های ۱ دقیقه تا ۱۲ ساعت
<code>get_trades()</code> , <code>get_live_trades()</code>	معاملات ریز تاریخی و امروز
<code>get_detail()</code> , <code>get_info()</code> , <code>get_stats()</code>	جزئیات، اطلاعات و آمار TSETMC نماد

تابع	کاربرد
<code>get_shareholders()</code>	سهامداران عمده فعلی یا تاریخی
<code>get_capital_increase()</code>	تاریخچه افزایش سرمایه
<code>get_price_adjustments()</code> , <code>get_latest_price_adjustment()</code>	رخدادهای تعدیل قیمت
<code>get_introduction()</code>	فقط سازگاری قدیمی؛ همیشه <code>UnsupportedDataSourceError</code>

داده زنده، سفارش و تحلیل بازار

تابع	کاربرد
<code>get_market_snapshot()</code>	خام و اتمیک کل بازار snapshot
<code>get_market_client_type()</code>	حقیقی/حقوقی bulk کل بازار
<code>get_live_market()</code> , <code>get_live_symbol()</code>	نمای زنده ادغام شده بازار یا یک نماد
<code>get_order_book()</code> , <code>get_queue()</code>	پنج سطح سفارش و صف زنده
<code>get_order_book_history()</code> , <code>get_queue_history()</code>	تاریخچه بازسازی شده سفارش و صف
<code>get_market_messages()</code>	پیامهای ناظر بازار
<code>get_instrument_state_changes()</code>	تغییر وضعیت ابزارها
<code>get_market_overview()</code>	نمای کلی رسمی بازار
<code>get_market_breadth()</code>	و شمار نمادهای مثبت/منفی A/D، breadth
<code>get_sector_flow()</code>	و جریان پول به تفکیک صنعت breadth
<code>watch_market()</code> , <code>MarketWatcher</code>	پایش افزایشی بازار و تولید <code>MarketEvent</code>

شاخص ها و تحلیل صنایع

تابع	کاربرد
<code>list_industry_indices()</code>	فهرست ۴۵ شاخص صنعت و وضعیت فعلی آنها
<code>get_industry_members()</code>	اعضای رسمی یک شاخص با اتصال دقیق <code>InsCode</code> به داده زنده
<code>get_industry_snapshot()</code>	یک ردیف تحلیلی برای هر صنعت: شاخص، breadth، معامله، حقیقی و صف
<code>get_industry_history()</code>	تاریخچه روزانه خود شاخص صنعت بدون حجم ساختگی
<code>get_industry_members_history()</code>	تاریخچه کوتاه همه اعضای فعلی صنعت در قالب long-form
<code>get_industry_intraday()</code>	مشاهدات خام یا کندل های درون روزی شاخص صنعت
<code>rank_industries()</code>	رتبه بندی صنایع بر اساس بازده، breadth، ارزش یا جریان پول

تاریخچه محلی و فاندمنتال

تابع	کاربرد
<code>save_market_snapshot()</code> , <code>load_market_snapshots()</code>	ذخیره و خواندن snapshot های SQLite
<code>get_live_market_history()</code>	تاریخچه نمای زنده ذخیره شده
<code>get_market_overview_history()</code>	تاریخچه overview رسمی
<code>get_market_snapshot_summary_history()</code>	خلاصه snapshot های ذخیره شده
<code>get_market_breadth_history()</code> , <code>get_sector_flow_history()</code>	تاریخچه تحلیل بازار و صنایع
<code>record_market_event()</code> , <code>get_market_event_history()</code>	ثبت و replay رخداد های watcher
<code>archive_market_records()</code>	آرشیو batch رکوردهای مستقل
<code>get_market_messages_history()</code>	پیام های آرشیو شده بازار
<code>get_instrument_state_changes_history()</code>	تغییر وضعیت های آرشیو شده
<code>check_market_history()</code>	بررسی سلامت و سازگاری فایل SQLite
<code>get_market_fundamentals()</code>	بازار EPS و P/E snapshot
<code>get_market_fundamentals_history()</code>	تاریخچه EPS/P/E بدون look-ahead

فهرست ابزارها و بازارها

تابع	کاربرد
<code>get_symbols()</code>	فهرست سهام، حق تقدم، صندوق، اوراق و اختیار
<code>list_options()</code>	فهرست قراردادهای اختیار فعال
<code>get_options_chain()</code>	زنجیره call/put یک دارایی پایه
<code>list_etfs()</code>	ETF ها NAV و discount/premium
<code>list_funds()</code> , <code>list_listed_funds()</code>	صندوق های ثبت شده و ابزارهای بورسی دقیق
<code>list_bonds()</code>	اوراق بدهی همراه سررسید
<code>list_indices()</code> , <code>get_index_companies()</code>	های بالا پیشنهاد می شوند API قدیمی شاخص ها و اعضای شاخص؛ برای صنعت API
<code>get_currency()</code>	ارز و سکه از API قدیمی TGJU

اخزا و درآمد ثابت

تابع	کاربرد
<code>parse_treasury_maturity()</code>	استخراج سررسید از نماد اخزا
<code>day_count_fraction()</code>	محاسبه فاصله زمانی با day-count convention

تابع	کاربرد
<code>treasury_yield()</code>	بازده اخزا از قیمت و سررسید
<code>bond_price()</code> , <code>yield_to_maturity()</code>	قیمت اوراق و حل YTM
<code>bond_analytics()</code>	duration, convexity و DV01
<code>build_yield_curve()</code>	ساخت شیء <code>YieldCurve</code> از <code>node</code> ها
<code>get_ifb_yield_table()</code>	جدول مرجع YTM فرابورس
<code>get_treasury_yields()</code>	اخزا و تحلیل بازده snapshot
<code>get_treasury_yield_history()</code>	تاریخچه YTM اخزا
<code>get_yield_curve()</code> , <code>get_yield_curve_history()</code>	منحنی بازده زنده و تاریخی

اختیار معامله

تابع	کاربرد
<code>black_scholes_price()</code>	قیمت بلک-شولز اروپایی
<code>black_scholes_greeks()</code>	Delta, Gamma, Vega, Theta و Rho
<code>option_price_bounds()</code>	کرانهای بدون آربیتراژ
<code>implied_volatility()</code>	حل نوسان ضمنی
<code>get_option_market()</code>	اتمیک بازار اختیار snapshot
<code>analyze_option_chain()</code>	و نقدشوندگی IV, Greeks, parity
<code>option_put_call_ratios()</code>	حجم، ارزش و موقعیت باز PCR
<code>get_option_history()</code>	تاریخچه قرارداد اختیار
<code>save_option_snapshot()</code> , <code>load_option_snapshots()</code>	ذخیره و خواندن snapshotهای JSON

هویت ابزار و ابزارهای کمکی

تابع	کاربرد
<code>resolve_instrument()</code>	تبدیل نماد یا <code>InsCode</code> به <code>InstrumentRef</code> دقیق
<code>validate_ins_code()</code>	اعتبارسنجی شناسه ابزار
<code>normalize_instrument_text()</code>	یکسان سازی ی/ک و فاصله های فارسی

برای signature و همه ورودی های هر تابع به مرجع [تفصیلی همه توابع](#) مراجعه کنید.

فهرست مطالب

- نصب و به‌روزرسانی
- شروع سریع
- راهنمای توابع پرکاربرد
- قراردادهای مهم داده
- حل دقیق هویت نماد
- قیمت و حقیقی/حقوقی
- معاملات ریز
- سفارش و صف
- و تحلیل کل بازار Watcher
- شاخص‌ها و تحلیل صنایع
- تاریخچهٔ محلی SQLite
- فاندمنتال، صندوق و تعدیل قیمت
- اخزا و درآمد ثابت
- اختیار معامله
- سایر API‌های بازار
- تنظیمات و خطاها
- مرجع تفصیلی همهٔ توابع
- مثال‌های کاربردی
- تست و مشارکت

نصب و به‌روزرسانی

```
pip install algotik-tse
```

برای ارتقا به آخرین نسخه:

```
python -m pip install --upgrade algotik-tse
```

نیازمندی نسخهٔ فعلی: **3.8** Python تا **3.14**.

برای نصب نسخهٔ توسعه:

```
git clone https://github.com/mohsenalipour/algotik_tse.git
cd algotik_tse
python -m pip install -e ".*[dev]"
```

شروع سریع

```
import algotik_tse as att
```

نیاز شما	تابع پیشنهادی
تاریخچه قیمت تعدیل شده	<code>att.get_history("شتران")</code>
حقیقی/حقوقی یک نماد	<code>att.get_client_type("شتران")</code>
اطلاعات زنده یک نماد	<code>att.get_live_symbol("شتران")</code>
لحظه ای کل بازار snapshot	<code>att.get_market_snapshot()</code>
کندل های اینترادی	<code>att.get_intraday("شتران", interval="5min")</code>
معاملات ریز امروز	<code>att.get_live_trades("شتران")</code>
پنج سطح سفارش	<code>att.get_order_book("شتران")</code>
تصویر تحلیلی صنایع	<code>att.get_industry_snapshot()</code>
رتبه بندی صنایع	<code>att.rank_industries(metric="breadth")</code>
فهرست نمادها و ابزارها	<code>att.get_symbols()</code>
قیمت ارز و سکه	<code>att.get_currency("dollar")</code>
صندوق های ETF با NAV	<code>att.list_etfs()</code>
اخزا و YTM	<code>att.get_treasury_yields()</code>
زنجیره و تحلیل اختیار	<code>att.get_option_market()</code> و <code>att.analyze_option_chain()</code>

اولین دریافت: تاریخچه قیمت

```
prices = att.get_history(
    "شتران",
    start="1403-01-01",
    end="1403-03-31",
    progress=False,
)
print(prices.tail())
```

خروجی نماینده:

J-Date	Open	High	Low	Close	Volume
1403-03-26	2630	2680	2605	2668	82471422
1403-03-27	2670	2715	2641	2692	70938510
1403-03-28	2695	2734	2670	2718	93612045

قیمت‌ها در حالت پیش‌فرض تعدیل می‌شوند. برای داده خام از `auto_adjust=False` و برای ستون‌های کامل‌تر از `output_type="full"` استفاده کنید.

چند کاربرد رایج در یک نگاه

```
# حقیقی/حقوق ۳۰ روز اخیر
client_type = att.get_client_type("فملی", limit=30, progress=False)

# است opt-in امروز؛ این رفتار observation تاریخچه به‌همراه
prices_today = att.get_history(
    "فملی", limit=20, include_today=True, progress=False
)

# نمای زنده نماد و قدرت خریدار
live = att.get_live_symbol("فملی", fallback="none")
print(live[["Symbol", "Last", "Close", "IndividualPower"]])

# پنج سطح سفارش، صف و معاملات امروز
book = att.get_order_book("فملی")
queue = att.get_queue("فملی", side="both", strict=True)
today_trades = att.get_live_trades("فملی")

# ابزارهای بازار
etfs = att.list_etfs()
funds = att.list_funds(fund_type="fixed_income")

# اخزا و اختیار معامله
treasuries = att.get_treasury_yields(min_volume=1)
options = att.get_option_market(underlying="خودرو")
analytics = att.analyze_option_chain(options, risk_free_rate=0.30)
```

خروجی نماینده `get_live_symbol()` در زمان بازار:

Symbol	Last	Close	IndividualPower
0 1.31		73950	74200 فملی

مقادیر مثال‌های متصل به شبکه «خروجی نماینده» هستند و با زمان بازار تغییر می‌کنند. مثال‌های ریاضی deterministic هستند و در تست‌های آفلاین کنترل می‌شوند. ستون‌های `Last` و `Close` در feed زنده به‌ترتیب «آخرین معامله» و «قیمت پایانی» هستند؛ تفاوت نام‌گذاری live و history در بخش بعد آمده است.

سند مرجع واحد پروژه است. اگر تازه شروع کرده‌اید، بخش راهنمای توابع پرکاربرد را بخوانید؛ مرجع تفصیلی همه [README.md](#) ورودی، خروجی و قرارداد هر تابع است و لازم نیست از ابتدا تا انتها خوانده شود، signature دقیق lookup توابع برای

راهنمای توابع پرکاربرد

این بخش برای استفاده روزمره است. ورودی‌های هر تابع کنار همان تابع توضیح داده شده‌اند؛ پس از آن، مرجع تخصصی تمام APIها قرار دارد.

get_history() — تاریخچه قیمت

```
get_history(
    symbol="", start=None, end=None, limit=0,
    raw=False, auto_adjust=True, output_type="standard",
    date_format="jalali", progress=True, save_to_file=False,
    dropna=True, adjust_volume=False, return_type=None,
    ascending=True, save_path=None, include_today=False,
    *, ins_code=None, asset_type="auto", **kwargs,
)
```

ورودی	گزینه‌ها و معنی
symbol	نماد فارسی یا فهرست نمادها؛ مانند "فملی" یا ["فملی", "شتران"] .
start , end	ابتدا و انتهای شامل بازه؛ تاریخ شمسی 1403-01-01 یا میلادی 2024-03-20 .
limit	تعداد آخرین ردیف‌های معاملاتی؛ 0 یعنی همه تاریخچه موجود.
raw	با True نام ستون‌ها به فرمت نزدیک TSETMC برمی‌گردد.
auto_adjust	پیش‌فرض True ؛ OHLC را با سری تعدیل‌شده بازسازی می‌کند.
output_type	و اطلاعات بیشتر Value , No. , Final برای "full" یا OHLCV برای "standard"
date_format	."both" یا "gregorian" ، "jalali"
progress	نمایش پیام پیشرفت؛ روی داده خروجی اثری ندارد.
save_to_file	ذخیره CSV را فعال می‌کند.
save_path	پوشه مقصد CSV؛ نام فایل از نماد ساخته می‌شود.
dropna	در خروجی چندنمادی ستون‌های کاملاً تهی را حذف می‌کند.
adjust_volume	حجم را نیز متناسب با ضریب افزایش سرمایه تعدیل می‌کند.
return_type	["simple", "Close", 5] یا فرم سفارشی مانند "both" ، "log" ، "simple"
ascending	جدید به قدیمی False قدیمی به جدید؛ True

ورودی	گزینه‌ها و معنی
<code>include_today</code>	با <code>observation = True</code> معتبر امروز را به صورت <code>opt-in</code> اضافه/جایگزین می‌کند.
<code>ins_code</code>	شناسه دقیق ابزار؛ برای نمادهای تکراری و <code>production</code> پیشنهاد می‌شود.
<code>asset_type</code>	<code>"bond"</code> ، <code>"index"</code> ، <code>"equity"</code> نوع ابزار مانند <code>hint</code> یا <code>"auto"</code> .
<code>**kwargs</code>	فقط <code>alias</code> های قدیمی مستند مانند <code>tse_format</code> ، <code>values</code> ، <code>stock</code> ؛ کلید ناشناخته API عمومی نیست.

خروجی: `DataFrame` تک‌نمادی یا `DataFrame` با ستون‌های `MultilIndex` برای چند نماد.

`get_client_type()` – حقیقی/حقوقی

```
get_client_type(
    symbol="", start=None, end=None, limit=0, raw=False,
    output_type="standard", date_format="jalali", progress=True,
    save_to_file=False, dropna=True, ascending=True, save_path=None,
    include_today=False, *, ins_code=None, asset_type="auto", **kwargs,
)
```

ورودی	گزینه‌ها و معنی
<code>symbol</code> ، <code>ins_code</code> ، <code>asset_type</code>	انتخاب نماد؛ قواعد آن‌ها مانند <code>get_history()</code> است.
<code>start</code> ، <code>end</code> ، <code>limit</code>	بازه یا تعداد آخرین روزهای معاملاتی.
<code>raw</code>	خروجی نزدیک به <code>schema</code> خام <code>provider</code> .
<code>output_type</code>	سرانه و قدرت را نیز نگه می‌دارد، <code>value = "full"</code> ؛ حالت کامل یا <code>"standard"</code> .
<code>date_format</code>	<code>"both"</code> یا <code>"gregorian"</code> ، <code>"jalali"</code> .
<code>include_today</code>	ردیف حقیقی/حقوقی امروز را با <code>provenance</code> و برچسب برآوردی بودن اضافه می‌کند.
<code>progress</code> ، <code>dropna</code> ، <code>ascending</code>	نمایش پیشرفت، حذف ستون تهی چندنمادی و ترتیب زمانی.
<code>save_to_file</code> ، <code>save_path</code>	ذخیره CSV و پوشه مقصد.
<code>**kwargs</code>	<code>limit=30</code> به جای <code>values=30</code> های قدیمی مانند <code>alias</code> .

خروجی: `DataFrame` روزانه تعداد/حجم/ارزش خریدوفروش حقیقی و حقوقی و شاخص‌های مشتق‌شده.

`get_live_symbol()` و `get_live_market()` – داده زنده

```
get_live_symbol(symbol=None, *, ins_code=None, fallback="none")
get_live_market(symbol=None, *, strict=False)
```


گزینه‌ها و معنی	تابع/ورودی
یک نماد فارسی یا <code>InsCode</code> .	<code>get_live_symbol.symbol</code>
شناسه دقیق keyword-only؛ بر selector مبهم ترجیح دارد.	<code>get_live_symbol.ins_code</code>
نقطه‌ای استفاده می‌کند endpoint در نبود نماد از <code>"point"</code> MarketWatch فقط <code>"none"</code> .	<code>fallback</code>
برای کل بازار، یک نماد یا فهرست نمادها <code>None</code> .	<code>get_live_market.symbol</code>
با <code>False</code> نماد گم‌شده در <code>attrs['missing_selectors']</code> ثبت می‌شود؛ با <code>True</code> خطا می‌دهد.	<code>strict</code>

خروجی: `get_live_symbol()` یک `DataFrame` یک‌ردیفی و `get_live_market()` یک `DataFrame` فیلترشده یا کل بازار می‌دهد. ستون‌های مهم شامل `Last`, `Close`, `IndividualPower`, `EstimatedNetIndividualFlow`, `SpreadBps` و `L5Imbalance` هستند.

get_market_snapshot() و feed – get_market_client_type() خام bulk

```
get_market_snapshot()
get_market_client_type()
```

این دو تابع ورودی کاربری ندارند. `get_market_snapshot()` یک `dict` شامل `stocks`, `order_book`, زمان بازار و `metadata` تازگی می‌دهد. `get_market_client_type()` یک `bulk DataFrame` حقیقی/حقوقی کل بازار برمی‌گرداند. برای مصرف معمول، `get_live_market()` نسخه ادغام‌شده و راحت‌تر است.

get_intraday() – تیک و کندل

```
get_intraday(
    symbol="شتران", interval="1min",
    start=None, end=None, progress=True, **kwargs,
)
```

گزینه‌ها و معنی	ورودی
نماد فارسی.	<code>symbol</code>
<code>"1m"</code> , <code>"tick"</code> , <code>"1min"</code> , <code>"5min"</code> , <code>"15min"</code> , <code>"30min"</code> , <code>"1h"</code> , <code>"4h"</code> , <code>"12h"</code> alias؛ <code>"60min"</code> , <code>"240m"</code> , <code>"720"</code> نیز پذیرفته می‌شوند.	<code>interval</code>
بدون مقدار، داده امروز؛ با تاریخ، <code>snapshot</code> ‌های تاریخی بازه.	<code>start</code> , <code>end</code>
نمایش پیشرفت.	<code>progress</code>
فقط سازگاری نام‌های قدیمی.	<code>**kwargs</code>

خروجی: در حالت tick داده معامله و در حالت candle ستون‌های `Open, High, Low, Close, Volume, TradeCount`. این API قدیمی برای ورودی نامعتبر ممکن است پیام چاپ کند و `None` بدهد.

فهرست ابزارها — `get_symbols()`

```
get_symbols(
    bourse=True, farabourse=True, payeh=True,
    haghe_taqadom=False, sandogh=False, bonds=False,
    options=False, mortgage=False, commodity=False, energy=False,
    payeh_color=None, output="dataframe", progress=True, **kwargs,
)
```

ورودی	گزینه‌ها و معنی
<code>bourse</code> , <code>farabourse</code> , <code>payeh</code>	بازارهای سهام که به صورت پیش فرض فعال‌اند.
<code>haghe_taqadom</code>	افزودن حق تقدم‌ها.
<code>sandogh</code>	افزودن صندوق‌ها.
<code>bonds</code>	افزودن اوراق بدهی.
<code>options</code>	افزودن اختیار معامله‌ها.
<code>mortgage</code>	افزودن اوراق تسهیلات مسکن.
<code>commodity</code>	افزودن گواهی‌های کالایی.
<code>energy</code>	افزودن ابزارهای انرژی.
<code>payeh_color</code>	"قرمز", "نارنجی", "زرد" یا فهرستی از آن‌ها.
<code>output</code>	یا خروجی فهرستی قدیمی <code>"dataframe"</code>
<code>progress</code>	نمایش پیشرفت.
<code>**kwargs</code>	نام‌های انگلیسی قدیمی فیلترها.

خروجی: فهرست ابزارها با هویت، نماد، نام، بازار و نوع دارایی.

ارز و سکه — `get_currency()`

```
get_currency(
    name="", start=None, end=None, limit=0,
    output_type="standard", date_format="jalali", progress=True,
    save_to_file=False, dropna=True, return_type=None,
    ascending=True, save_path=None, **kwargs,
)
```

ورودی	گزینه‌ها و معنی
name	یک نام یا فهرست؛ مانند "dollar", "euro", "seke", "نیم سکه".
start, end, limit	بازه یا آخرین تعداد ردیف.
output_type	قدیمی API یا حالت کامل پشتیبانی‌شده "standard".
date_format	"jalali", "gregorian", "both".
return_type	محاسبه بازده ساده/لگاریتمی.
progress, dropna, ascending	پیشرفت، خروجی چندارزی و ترتیب زمانی.
save_to_file, save_path	ذخیره CSV و پوشه مقصد.
**kwargs	های سازگاری قدیمی alias.

خروجی: `DataFrame[Open, High, Low, Close]`؛ برای چند ارز ستون‌ها MultiIndex می‌شوند.

get_trades() و get_live_trades() — معاملات ریز

```
get_trades(
    symbol=None, *, ins_code=None, start=None, end=None,
    include_canceled=False, max_requests=None, raw=False, progress=True,
)
get_live_trades(
    symbol=None, *, ins_code=None,
    include_canceled=False, max_requests=None, raw=False, progress=True,
)
```

ورودی	گزینه‌ها و معنی
symbol, ins_code	انتخاب نماد با نام یا شناسه دقیق.
start, end	بازه تاریخی شامل دو سر؛ فقط در <code>get_trades()</code> .
include_canceled	نگه‌داشتن معاملات ابطالی.
max_requests	سقف سخت تعداد درخواست‌ها؛ برای بازه‌های بزرگ حتماً تعیین کنید.
raw	افزودن ستون‌های audit نزدیک provider.
progress	نمایش پیشرفت.

خروجی: `DataFrame` معاملات با `TradeNo`, `Timestamp`, `Price`, `Volume`, `Value`, `Canceled`, `Source`.

get_order_book() و get_queue() – سفارش و صف زنده

```
get_order_book(symbol=None, *, selector_strict=False)
get_queue(symbol=None, side="both", strict=True, *, selector_strict=False)
```

ورودی	گزینه‌ها و معنی
symbol	برای کل بازار، یک نماد یا فهرست نمادها None
selector_strict	گم‌شده یا مبهم را به خطا تبدیل می‌کند selector
side	در get_queue(): "buy", "sell", "both"
strict	فقط صف قطعی مبتنی بر قیمت مجاز و level یک را نگه می‌دارد.

خروجی: order book به شکل long با پنج level؛ queue با سمت، قیمت، حجم، تعداد سفارش و ارزش صف.

list_etfs(), list_funds(), list_bonds() و اختیارها

```
list_etfs(progress=True)
list_funds(fund_type=None, progress=True, *, listed_only=False)
list_bonds(progress=True)
list_options(underlying=None, progress=True)
get_options_chain(underlying, fetch_oi=False, progress=True)
```

تابع/ورودی	گزینه‌ها و معنی
progress	در همه این توابع فقط نمایش پیشرفت را کنترل می‌کند.
fund_type	فهرست چند نوع نیز مجاز. "equity", "fixed_income", "mixed", "commodity", "market_maker", "venture", "project", "real_estate", "private", "fund_of_funds". برای همه یا None است.
listed_only	در list_funds() فقط ابزارهای قابل معامله را برمی‌گرداند؛ هم‌زمان با fund_type مجاز نیست.
underlying	در list_options() فیلتر اختیاری و در get_options_chain() دارایی پایه اجباری.
fetch_oi	در زنجیره اختیار، دریافت Open Interest و metadata تکمیلی را فعال می‌کند و ممکن است درخواست‌های بیشتری بسازد.

خروجی: همه DataFrame هستند، به جز get_options_chain() که dict شامل calls, puts, underlying_price, expiry_dates و market_time می‌دهد.

قراردادهای مهم داده

نوع خروجی

همه خروجی‌ها DataFrame نیستند:

خانواده	نوع خروجی
قیمت، حقیقی/حقوقی، trades، order book، fundamentals و فهرست ابزارها	<code>None</code> گاهی legacy های API یا در <code>pandas.DataFrame</code> هنگام خطا
<code>market_watch()</code> / <code>get_market_snapshot()</code>	و زمان <code>metadata</code> ، <code>order_book</code> ، <code>stocks</code> شامل <code>dict</code> مشاهده
<code>get_options_chain()</code>	<code>calls</code> و <code>puts</code> شامل <code>dict</code>
<code>resolve_instrument()</code>	شیء <code>ImmutableRef</code> از نوع
<code>watch_market()</code> / <code>MarketWatcher</code>	<code>MarketEvent</code> از iterator
<code>get_yield_curve()</code> / <code>build_yield_curve()</code>	شیء <code>YieldCurve</code>
توابع Black-Scholes	عدد، tuple یا <code>dict</code> مطابق تابع

برای DataFrame های جدید، خروجی خالی همان `columns` و `dtypes` خروجی غیرخالی را نگه می‌دارد. `metadata` های مهم مانند منبع، `freshness`، پوشش، `partial` بودن و بودجه درخواست در `df.attrs` قرار می‌گیرند:

```
df = att.get_live_market("فملی")
print(df.attrs)
```

```
{
  'trade_date': datetime.date(...),
  'exchange_time': '12:28:41',
  'fetched_at': Timestamp(..., tz='Asia/Tehran'),
  'is_realtime_fresh': True,
  'is_partial': False,
  'missing_selectors': []
}
```

قیمت پایانی و آخرین معامله

- در feed زنده: `Last` آخرین قیمت معامله و `Close` قیمت پایانی TSETMC است.
- در تاریخچه سهام: `Close` آخرین قیمت معامله روز است؛ `Final` قیمت پایانی است و فقط در `output_type="full"` دیده می‌شود.
- با `auto_adjust=True` (پیش‌فرض)، `OHLC` و `Final` در صورت حضور، تعدیل شده‌اند.
- با `Close`، `auto_adjust=False` و `Final` خام‌اند و `Adj Close` نیز ارائه می‌شود.

- هنگام **Last** ، **include_today=True** زنده به **Close** تاریخچه و **Close** زنده به **Final** تاریخچه نگاشت می‌شود. بنابراین صرفاً بر اساس نام ستون بین **live** و **history join** ننزید.

تاریخ، ردیف امروز و freshness

- می‌پذیرند (2024-07-22) یا میلادی (1403-05-01) شمسی ISO شامل دو سر بازه‌اند و تاریخ **end** و **start** رفتار قبلی حفظ شده است: **include_today=False** هیچ درخواست زنده اضافه‌ای انجام نمی‌دهد.
- قدیمی snapshot . می‌کند **append/replace** معتبر همان روز معاملاتی را **observation** فقط **include_today=True** خواهد داشت **live_is_realtime_fresh=False** همان روز ممکن است برای تکمیل تاریخچه پذیرفته شود ولی
- توقف نماد، نبود معامله، نبود هویت قطعی یا شکست **live** باعث جعل ردیف امروز نمی‌شود؛ تاریخچه موفق برمی‌گردد و هشدار/attrs دلیل را نشان می‌دهند.
- مانند **server-side** ادعا نمی‌کند. تاریخچه **backfill** محلی فقط از زمان ضبط شما پوشش دارد و **history** قرارداد جداگانه دارد **price/trades/order-book**.

تقارن API زنده/تاریخی:

داده	زنده	تاریخی
قیمت و نمای نماد	<code>get_live_symbol</code> , <code>get_live_market</code>	<code>get_history(include_today=...)</code>
حقیقی/حقوقی	<code>get_live_market</code> , <code>get_market_client_type</code>	<code>get_client_type(include_today=...)</code>
معاملات ریز	<code>get_live_trades</code>	<code>get_trades</code>
پنج سطح سفارش	<code>get_order_book</code>	<code>get_order_book_history</code>
صف	<code>get_queue</code>	<code>get_queue_history</code>
فاندامنتال snapshot	<code>get_market_fundamentals</code>	<code>get_market_fundamentals_history</code>
اخزا و YTM	<code>get_treasury_yields</code>	<code>get_treasury_yield_history</code>
منحنی بازده	<code>get_yield_curve</code>	<code>get_yield_curve_history</code>
اختیار	<code>get_option_market</code>	<code>get_option_history</code> و snapshot های <code>save_option_snapshot</code> / <code>load_option_snapshots</code>
overview/breadth/sector/message/state	متناظر live های helper	یا <code>archive_to</code> پس از <code>*_history</code> های helper snapshot

تقارن به معنی یکسان بودن منبع نیست: بعضی **history** ها **server-side** هستند و بعضی فقط **observation** های ذخیره شده کاربر را می‌خوانند. **NoBackfill** , **Source** و **coverage** را بررسی کنید.

ذخیره در CSV

در API های تاریخی **save_path** ، **legacy** مسیر پوشه است، نه نام فایل. نام فایل از نماد/دارایی ساخته می‌شود:

```
att.get_history(
    "فملی", save_to_file=True, save_path="exports", progress=False
)
# exports/فملی.csv
```

source boundary

درخواست‌های runtime فقط به providerهای پشتیبانی‌شده TSETMC، صفحه مرجع YTM فرابورس ایران (ifb.ir) و API قدیمی ارز/سکه TGJU محدودند. redirect به origin دیگر پیش از درخواست دوم رد می‌شود. هیچ API کدال در این پکیج وجود ندارد.

قطعی و قابل استناد منتشر نمی‌کند. اختلاف قیمت تعدیل‌شده و تعدیل‌نشده **معادل DPS**، برای رخداد تعدیل قیمت TSETMC. استنتاج نمی‌کند DPS **سود نقدی نیست** و کتابخانه از آن

حل دقیق هویت نماد

نمادهای تکراری، ابزار منقضی، حق تقدم، اختیار و اوراق می‌توانند نام مشابه داشته باشند. APIهای جدید از **InstrumentRef** و **InsCode** استفاده می‌کنند:

```
ref = att.resolve_instrument("فملی", asset_type="equity")
print(ref)
```

خروجی نماینده:

```
InstrumentRef(
    ins_code='35425587644337450', symbol='فملی',
    name='ملی صنایع مس ایران', asset_type='equity',
    is_active=True, provenance='tsetmc_search_exact', selector='فملی'
)
```

برای اجرای قطعی در **ins_code**، production، بدهید:

```
prices = att.get_history(
    ins_code="35425587644337450", asset_type="equity", progress=False
)
live = att.get_live_symbol(ins_code="35425587644337450")
```

را می‌پذیرد و نتیجه را به صورت رشته برمی‌گرداند؛ ASCII عدد صحیح مثبت دقیق یا رشته ۱ تا ۲۰ رقم **validate_ins_code()** بیش از یک نتیجه معتبر داشته باشد، selector رقم فارسی/عربی، صفر و مقدار مبهم پذیرفته نمی‌شوند. اگر، **bool**، **float** کتابخانه حدس نمی‌زند:

```
try:
    att.resolve_instrument("نماد تکراری")
except att.AmbiguousSymbolError as exc:
    print(exc)
```

AmbiguousSymbolError: selector matched more than one instrument; pass ins_code

گزینه‌های اصلی:

```
att.resolve_instrument(
    selector=None,
    ins_code=None,
    asset_type="auto",
    snapshot=None,
    require_active=True,
)
```

- دوباره را می‌دهد fetch بدون resolve امکان `snapshot`
- برای تاریخچه ابزار منقضی مناسب است `require_active=False`
- هویتی fuzzy join و فاصله‌های متداول را یکسان می‌کند، اما `ک/ک`، `ی/ی` اختلاف `normalize_instrument_text()` انجام نمی‌دهد.

قیمت و حقیقی/حقوقی؛ تاریخچه و زنده

`get_history()`

```
att.get_history(
    symbol="فملی", start=None, end=None, limit=0,
    raw=False, auto_adjust=True, output_type="standard",
    date_format="jalali", progress=True, save_to_file=False,
    dropna=True, adjust_volume=False, return_type=None,
    ascending=True, save_path=None, include_today=False,
    ins_code=None, asset_type="auto",
)
```

```
history = att.get_history(
    "فملی", limit=10, include_today=True,
    output_type="full", progress=False,
)
print(history.tail(2))
print(history.attrs["include_today_appended"])
```


خروجی نماینده:

	Open	High	Low	Close	Final	Volume	No.	Value
Date								
1405-06-02	73100	74600	72800	74200	73950	1820043	3912	1.35e+11
1405-06-03	74000	74900	73600	74700	74420	814220	1830	6.06e+10

['فملی']

در `standard` ستون‌ها `Open, High, Low, Close, Volume` هستند. `full` ستون‌های `Final, No., Value` و اطلاعات تقویم/Ticker را نیز اضافه می‌کند. `raw=True` فرمت TSE را برمی‌گرداند. `return_type` یکی از `simple`، `log`، `both` یا فرم سفارشی مانند `['simple', 'Close', 5]` است. برای چند نماد، `DataFrame` با `MultilIndex` ستونی برمی‌گردد.

`get_client_type()`

```
client = att.get_client_type(
    "فملی", limit=30, include_today=True,
    output_type="full", progress=False,
)
print(client.tail(1))
```

خروجی نماینده ردیف امروز:

	No_buy_retail	Vol_buy_retail	No_sell_retail	Vol_sell_retail	Power_retail	Is_partial
Date						
1403-05-07	1284	920000	991	701000	1.24	True

ردیف امروز `volume/count` را از `ClientTypeAll` می‌گیرد. `value`های امروز در صورت نیاز با `VWAP` بازار برآورد می‌شوند و ستون‌های `Value_source` / `Is_estimated` در خروجی `opt-in` مشخص می‌کنند که مقدار رسمی یا برآوردی است. رفتار پیش‌فرض و `schema` قدیمی بدون `include_today` تغییر نکرده است.

live کل بازار و یک نماد

```
snapshot = att.get_market_snapshot()      # dict سازگار با market_watch()
live_all = att.get_live_market()          # DataFrame ادغام‌شده
live_one = att.get_live_market("فملی")
point = att.get_live_symbol("فملی", fallback="none")
```

می‌کند. ستون‌های کلیدی `join` و بهترین سفارش‌ها را `client type`، قیمت `get_live_market()`

```
InsCode, Symbol, Name, Last, Close, PreviousClose, Volume, Value,
IndividualBuyVolume, IndividualSellVolume, LegalBuyVolume, LegalSellVolume,
IndividualPower, NetIndividualVolume, EstimatedNetIndividualFlow,
BidPrice1, AskPrice1, Spread, SpreadBps, L1Imbalance, L5Imbalance,
trade_date, exchange_time, fetched_at, is_realtime_fresh, is_partial
```

freshness در `get_live_market()` attr عمومی است، نه row-wise ستون audit schema/selection واقعی برای attrs هستند:

```
print(live_all.attrs.keys())
print(live_all.attrs["missing_selectors"])
```

```
dict_keys(['field_validity', 'source_schema_presence', 'migration', 'missing_selectors'])
[]
```

`get_live_symbol(..., fallback="none")` در MarketWatch، `StockNotFoundError` می‌دهد. دقیقاً یکی از `SnapshotSource` را امتحان می‌کند؛ ستون TSETMC نقطه‌ای endpoint فقط در آن حالت `fallback="point"` است. `PresentInMarketWatch`، `FallbackReason`، `market_watch` یا `closing_price_info_fallback` است. `IdentityVerified`، `has_trade_today` و `PriceActionable` قابلیت استفاده قیمت را شفاف می‌کنند.

پنج سطحی دارد long خروجی `get_order_book()`

InsCode	Symbol	Level	BidOrderCount	BidVolume	BidPrice	AskPrice	AskVolume	AskOrderCount
...	31		124000	74200	74100	180000	42	1

فملی

ستون‌های metadata آن شامل `trade_date`، `exchange_time`، `fetched_at`، `snapshot_age_seconds`، `is_partial` و `is_realtime_fresh`، `is_stale` است.

معاملات ریز

همان قرارداد را `get_live_trades()` استفاده می‌کند و TSETMC تاریخچه معاملات endpoint برای هر روز از `get_trades()` برای روز جاری ارائه می‌دهد:

```
trades = att.get_trades(
    "فملی",
    start="1403-05-01",
    end="1403-05-03",
    include_canceled=False,
    max_requests=10,
    raw=False,
    progress=False,
)

live_trades = att.get_live_trades(
    ins_code="35425587644337450",
    include_canceled=False,
    max_requests=1,
    progress=False,
)
```

استاندارد schema:

InsCode, Symbol, GregorianDate, JalaliDate, TradeNo, Time, Timestamp,
Price, Volume, Value, Canceled, Source

خروجی نماینده:

	TradeNo	Time	Price	Volume	Value	Canceled	Source
Timestamp							
...	1	09:01:01	50000	100	5000000	False	tsetmc_trade_history_lossless
...	2	09:01:01	50010	50	2500500	False	tsetmc_trade_history_lossless

را نیز نگه `RawCanceled` و `RawDate` ، `RawInsCode` ، `iSensVarP` ، `qTitNgJ` مانند provider فیلدهای خام `raw=True` های مهم `attrs` می‌دارد.

```
request_count, trade_request_count, max_requests, request_budget_scope,
partial, failures, source_coverage, reconciliation,
resolved_ins_code, resolved_symbol
```

- ممکن است هزینه جداگانه داشته باشد و resolver است؛ trade endpoint سقف سخت درخواست‌های `max_requests` مشخص می‌کند شمارش کل شناخته‌شده است یا نه `attrs`.
- شکست بخشی از روزها با `partial=True` و جزئیات `failures` برمی‌گردد؛ داده موجود دور ریخته نمی‌شود.
- رد می‌شود `DataParsingError` پاسخ، تاریخ یا فیلدهای عددی با `InsCode` در mismatch
- رکورد ابطالی فقط با `include_canceled=True` نگه داشته می‌شود.

سفارش و صف

سفارش زنده

```
book = att.get_order_book("فملی")
queue = att.get_queue("فملی", side="both", strict=True)
```

قیمت صف را با `strict=True` قطعی `False` است، نه `NA` صف را سه حالتی گزارش می‌کند؛ نبود شواهد کافی `get_queue()` بازار تطبیق می‌دهد state دامنه مجاز و

```
InsCode Symbol Side QueuePrice QueueVolume QueueOrders QueueValue
...      فملی   buy      73500      820000      163  6.027e+10
```

```
is_queue=True, is_strict=True, is_partial=False,
threshold_source='market_watch_price_limits', book_source='market_watch_live_snapshot'
```

تاریخچه پنج سطح سفارش

```
books = att.get_order_book_history(
    "فملی",
    limit=20,
    output_type="standard", # long یک ردیف در هر روز
    include_today=True,
    complete_only=True,
    max_requests=25,
    progress=False,
)

wide = att.get_order_book_history(
    "فملی", limit=10, output_type="wide", max_requests=25, progress=False
)
```

داده تاریخی `BestLimits` یک `delta stream` روزانه است. کتابخانه `state` هر روز را مستقل و به ترتیب `hEven/refID` بازسازی می‌کند؛ مقدار صفر پاک شدن `level` است. `schema long`:

```
InsCode, Symbol, Name, Date, GregorianDate, JalaliDate, Time, Timestamp,
DEven, hEven, refID, _sequence, Level,
BidOrderCount, BidVolume, BidPrice, AskPrice, AskVolume, AskOrderCount,
is_partial, is_complete, market_partial_status, is_reconstructed,
is_stale, record_type, source
```

خروجی نماینده:

Timestamp	Level	BidPrice	BidVolume	AskPrice	AskVolume	is_complete	source
2024-07-22 09:05:01+03:30	1	74100	180000	74200	124000	True	tsetmc_best_limits_history_reconstructed
2024-07-22 09:05:01+03:30	2	74050	95000	74250	88000	True	tsetmc_best_limits_history_reconstructed

برابر source زنده معتبر را با snapshot `include_today=True` ناقص را حذف می‌کند `complete_only=True`. سقف سخت است و `max_requests` اضافه یا جایگزین می‌کند `market_watch_live_snapshot`. پوشش ناقص را توضیح می‌دهند `attrs['request_count']` و `attrs['failed_requests']`. نام alias `get_orderbook_history` عینی `get_order_book_history` است.

تاریخچه صف

```
queues = att.get_queue_history(
    "فملی",
    limit=20,
    include_today=True,
    complete_only=True,
    side="both",
    strict=True,
    max_requests=25,
    progress=False,
)
```

schema:

```
InsCode, Symbol, Name, Date, GregorianDate, JalaliDate, Time, Timestamp,
DEven, hEven, Side, QueuePrice, QueueVolume, QueueOrders, QueueValue,
Pricelimit, is_strict, is_queue, is_partial, is_complete,
market_partial_status, book_state, is_crossed, is_preopen_or_stopped,
threshold_hEven, threshold_source, book_source, source
```

اگر static threshold تاریخی در دسترس نباشد، `threshold_source='unavailable'` و `is_queue=NA` می‌ماند. این رفتار برای backtest مهم است: نبود داده به «صف نبود» تبدیل نمی‌شود.

Watcher و تحلیل کل بازار

iterator بازار

```
for event in att.watch_market(
    symbol="فملی",
    interval=2,
    max_updates=5,
    notifications=("messages", "state"),
):
    print(event.kind, event.sequence, event.fetched_at, event.changed_inscodes)
```

شامل event. وجود ندارد `kind="update"`. است `initial`, `delta`, `heartbeat`, `resync` یکی از `MarketEvent.kind` است. `snapshot`، `persistence` و وضعیت `retry`، `cursor`، `level` تغییرکرده، `InsCode` تغییرکرده فهرست،

برای کنترل کامل:

```
watcher = att.MarketWatcher(
    symbol=None,
    interval=2,
    max_updates=None,
    include_initial=True,
    emit_heartbeats=True,
    notifications=("messages", "state"),
    notification_top=50,
    error_policy="retry",
    max_consecutive_retries=5,
    max_backoff=30,
    callback_error_policy="raise",
)
```

`resync` محدود `backoff` را نگه می‌دارد و در قطع ارتباط با `cursor`، استفاده می‌کند `MarketWatchInit/Plus` از `Watcher` هستند `state` و `messages` ها فقط `notification` می‌شود.

پیام، وضعیت و نمای بازار

```
messages = att.get_market_messages(flow=0, top=20, since_id=None)
states = att.get_instrument_state_changes(top=20, since_id=None)
overview = att.get_market_overview(flow=0)
```

پیام schema:

```
message_id, date, time, timestamp, title, description, flow
```

وضعیت schema:

```
event_id, date, time, timestamp, InsCode, Symbol, Name,
state_code, state, real_time, under_supervision, state_title
```

رسمی payload برمی‌گرداند؛ مجموعه ستون‌ها به `flow` را با ستون `overview` رسمی `raw` فیلدهای `get_market_overview()` وابسته است و به تعداد ثابتی از ستون‌ها متعهد نیست `provider`.

breadth و جریان صنایع

```
breadth = att.get_market_breadth(traded_only=True)
sectors = att.get_sector_flow(traded_only=True)
```

خروجی نماینده breadth:

instrument_count	advances	declines	unchanged	ad_difference	ad_ratio	total_volume	upper_limit_count
lower_limit_count	612	318	241	53	77	1.32	8.91e+09
18							42

ستون‌های breadth علاوه بر موارد بالا شامل `no_trade`, `missing_previous`, `missing_current_price` درصد صعود/نزول، `total_value`, `trade_date`, `exchange_time`, `fetch_at`, `is_realtime_fresh` است.

همراه با پوشش حقیقی/حقوقی ارائه می‌کند `SectorCode` را برای هر breadth همین `get_sector_flow()`:

SectorCode	instrument_count	advances	declines	client_coverage	net_individual_volume
estimated_net_individual_value	27	43	25	13	0.91
5.8e+10	market_vwap	False			820000

را `freshness` و `client_coverage`, `value_method`, `value_available` برآورد است؛ `estimated_net_individual_value` در استراتژی لحاظ کنید.

فیلترهای مشترک عبارت‌اند از `instrument_types` و `symbol`, `flow`, `sector`, `traded_only`, `include_base_market`. برای محاسبه چند خروجی روی یک مشاهده، `snapshot` را یک‌بار دریافت و به مسیر خصوصی `_snapshot` ندهید؛ API عمومی `save_market_snapshot()` یک `snapshot` اتمیک می‌سازد و `history derivation` را هم‌زمان نگه می‌دارد.

شاخص‌ها و تحلیل صنایع

صنعت در نسخه 1.2.0 دو مفهوم را از هم جدا می‌کند API

- **عضویت رسمی شاخص:** نمادهایی که endpoint رسمی `GetIndexCompany` برای همان شاخص برمی‌گرداند. توابع این فصل به‌طور پیش‌فرض از این universe استفاده می‌کنند.
- **گروه دیده‌بان:** دسته‌بندی سریع `SectorCode` در MarketWatch که مبنای `get_sector_flow()` است و الزاماً با اعضای رسمی شاخص برابر نیست.

پارامترهای اختصاصی این خانواده عبارت‌اند از `industry`, `industries`, `include_member_count`, `include_live`, `include_client_type`, `include_orderbook`, `include_empty`, `days`, `metric`, `top`, `refresh`, `max_workers` و ورودی‌های عمومی `interval` و `progress`, `start`, `end`, `limit`, `ascending` همان معنای توضیح‌داده‌شده در هر تابع را دارند.

هویت عضوها فقط با `InsCode` متصل می‌شود. نام مشابه یا جست‌وجوی fuzzy برای join استفاده نمی‌شود. همچنین بعضی شاخص‌ها تجمیعی‌اند و عضویت صنایع می‌تواند هم‌پوشانی داشته باشد؛ بنابراین جمع‌زدن ردیف‌های تمام صنایع، کل بازار بدون تکرار تولید نمی‌کند.

مسیر سریع برای کاربر معمولی

```
import algotik_tse as att

# شاخص صنعت؛ سریع و تنها با یک درخواست ۴۵
indices = att.list_industry_indices(progress=False)

# اعضای رسمی یک صنعت همراه داده جاری بازار
members = att.get_industry_members("فلزات اساسی", progress=False)

# snapshot تحلیلی همه صنایع همراه حقیقی/حقوقی
snapshot = att.get_industry_snapshot(progress=False)

# breadth پنج صنعت برتر از نظر
leaders = att.rank_industries(
    metric="breadth",
    top=5,
    progress=False,
)
```

فهرست شاخص‌های صنعت

```
indices = att.list_industry_indices(
    progress=False,
    include_member_count=True,
    refresh=False,
    max_workers=6,
)
```

تهی می‌مانند. با `HasMembers` و `MemberCount` پیش‌فرض و سریع است؛ در این حالت `include_member_count=False` عضویت را دور می‌زند `cache refresh=True` می‌شود `cache` عضویت رسمی همه صنایع دریافت و در حافظه، `True` درخواست‌های عضویت است concurrency و فقط سقف 1.16 عدد صحیح `max_workers`

schema:

```
IndustryName, IndustryNameEn, IndustryGroupCode, IndexInsCode,
IndexValue, IndexPreviousValue, DayHigh, DayLow,
IndexChange, IndexChangePct, MemberCount, HasMembers, ExchangeTime
```

نسخه‌های `list_indices()` درصد تغییر است. این نگاشت در `IndexChangePct` تغییر واحد شاخص و `IndexChange` برعکس بود و در 1.2.0 اصلاح شده است `<=1.1.3`.

اعضای دقیق یک شاخص

```
members = att.get_industry_members(
    industry="بانک",
    include_live=True,
    include_client_type=True,
    include_orderbook=True,
    progress=False,
    refresh=False,
)
```

ورودی `industry` می‌تواند نام فارسی، alias شناخته‌شده مانند `بانک`، صورت کامل مانند `شاخص صنعت بانکها` یا `IndexInsCode` باشد.

- ادغام می‌کند MarketWatch جاری snapshot اعضای رسمی را با `include_live=True`
- شده قابل reconcile قدرت خریدار و جریان حقیقی/حقوقی را اضافه می‌کند؛ فقط ردیف‌های `include_client_type=True` استفاده‌اند.
- `include_live=True` و صف تخمینی پنج سطح را اضافه می‌کند و به `spread`، `imbalance` `include_orderbook=True` نیاز دارد.
- می‌گیرد TSETMC عضویت را دوباره از `refresh=True`

ثابت خروجی schema:

```
IndustryName, IndustryIndexCode, InsCode, Symbol, Name,
SectorCode, Flow, MarketCode, InstrumentType,
PreviousClose, Open, High, Low, Close, Last, Change, ChangePct,
TradeCount, Volume, Value, SharesOutstanding, EstimatedMarketCap,
IndividualPower, NetIndividualVolume, EstimatedNetIndividualFlow,
ClientDataAvailable, BidPrice1, AskPrice1, SpreadBps,
L1Imbalance, L5Imbalance,
EstimatedBuyQueueVolume, EstimatedBuyQueueValue,
EstimatedSellQueueVolume, EstimatedSellQueueValue,
TradeDate, ExchangeTime, FetchedAt, IsRealtimeFresh, IsStale
```

ستون‌های مربوط به client type یا order book وقتی درخواست نشده‌اند nullable می‌مانند. `EstimatedMarketCap` حاصل `SharesOutstanding × Close` و `EstimatedNetIndividualFlow` برآورد مبتنی بر VWAP است؛ هیچ‌کدام وزن رسمی شاخص نیستند.

در `DataFrame.attrs` مواردی مانند `universe='exact_index_members'` , `membership_is_current` , `historical_membership_available` , `cache_hit` , `client_type_requested` , `orderbook_requested` و `membership_count` ثبت می‌شوند.

snapshot تحلیل صنعت

برای یک صنعت:

```
metals = att.get_industry_snapshot(
    industries="فلزات اساسی",
    include_client_type=True,
    include_orderbook=True,
    include_empty=False,
    progress=False,
    refresh=False,
    max_workers=6,
)
```

برای چند صنعت:

```
selected = att.get_industry_snapshot(
    industries=["بانک", "خودرو", "شیمیایی"],
    include_client_type=True,
    progress=False,
)
```

با `industries=None` تمام صنایع بررسی می‌شوند. `include_empty=False` شاخص‌هایی را که provider در آن لحظه عضو ندارند حذف می‌کند؛ نام آن‌ها در `attrs['empty_industries']` باقی می‌ماند. این تابع فقط یک MarketWatch bulk، حداکثر یک client-type bulk و عضویت‌های cache شده را مصرف می‌کند.

گروه‌های اصلی خروجی:

گروه	ستون‌ها
وضعیت شاخص	<code>IndexValue</code> , <code>IndexPreviousValue</code> , <code>IndexChange</code> , <code>IndexChangePct</code> , <code>IndexDayHigh</code> , <code>IndexDayLow</code>
breadth	<code>MemberCount</code> , <code>TradedCount</code> , <code>Advances</code> , <code>Declines</code> , <code>Unchanged</code> , <code>NoTrade</code> , <code>AdvanceDeclineRatio</code> , <code>AdvancePct</code> , <code>DeclinePct</code>
عملکرد اعضا	<code>EqualWeightReturn</code> , <code>MedianReturn</code> , <code>ReturnDispersion</code>
معاملات	<code>TotalTradeCount</code> , <code>TotalVolume</code> , <code>TotalValue</code> , <code>MarketValueSharePct</code> , <code>UpperLimitCount</code> , <code>LowerLimitCount</code>

گروه	ستون‌ها
حقیقی / حقوقی	<code>ClientCoveredCount</code> , <code>ClientCoverage</code> , <code>NetIndividualVolume</code> , <code>EstimatedNetIndividualValue</code> , <code>IndividualPower</code> , <code>FlowValueMethod</code>
سفارش و صف	<code>OrderBookCoveredCount</code> , <code>OrderBookCoverage</code> , <code>BuyQueueCount</code> , <code>BuyQueueValue</code> , <code>SellQueueCount</code> , <code>SellQueueValue</code>
زمان و تازگی	<code>TradeDate</code> , <code>ExchangeTime</code> , <code>FetchAt</code> , <code>IsRealtimeFresh</code> , <code>IsStale</code>

`IndividualPower` میانگین ساده بازده اعضای دارای قیمت معتبر است و بازده رسمی شاخص نیست `EqualWeightReturn`.
 های سازگار با client row از سرانۀ تجمیعی خرید حقیقی به سرانۀ تجمیعی فروش حقیقی ساخته می‌شود. جریان پول فقط از تازه و متعلق به روز order book را کنار آن کنترل کنید. صف‌ها فقط برای `ClientCoverage` حجم بازار جمع می‌شود؛ همیشه تهران محاسبه می‌شوند.

در , `memberships_may_overlap=True` , `membership_is_current=True` , `historical_membership_available=False` , تعداد cache hit/request و روش تخمین جریان پول ثبت می‌شود.

تاریخچه شاخص و اعضا

تاریخچه خود شاخص:

```
history = att.get_industry_history(
    industry="شیمیایی",
    start="1404-01-01",
    end=None,
    limit=120,
    ascending=True,
    progress=False,
)
```

یعنی `limit=0` هستند. فیلتر تاریخ ابتدا اعمال می‌شود و `YYYY-MM-DD / YYYYMMDD` تاریخ شمسی یا میلادی `start` و `end` جلسه را نگه می‌دارد. خروجی N بدون محدودیت؛ مقدار مثبت آخرین

است. منبع رسمی برای `IndustryName, IndustryIndexCode, TradeDate, JalaliDate, High, Low, Close, Change, ChangePct` `attrs['volume_available']=False` روزانه نمی‌دهد؛ ستون حجم مصنوعی ساخته نمی‌شود و volume شاخص صنعت است.

تاریخچه کوتاه همه اعضای فعلی:

```
member_history = att.get_industry_members_history(
    industry="خودرو",
    days=30,
    ascending=True,
    progress=False,
    refresh=False,
)
```

long-form رسمی است. خروجی payload و تعداد آخرین معاملاتی موجود در **1..30** عدد صحیح **days**

```
IndustryName, IndustryIndexCode, TradeDate, JalaliDate,
InsCode, Symbol, Name, Close, Last, Change, ChangePct,
TradeCount, Volume, Value
```

این history برای **اعضای فعلی** شاخص است، نه عضویت point-in-time. اگر ترکیب شاخص تغییر کرده باشد، survivorship bias محتمل است؛ به همین دلیل attrs صریح **point_in_time_membership=False** و **survivorship_bias_possible=True** دارد.

داده درون‌روزی شاخص صنعت

```
intraday = att.get_industry_intraday(
    industry="فلزات",
    interval="5min",
    progress=False,
)
```

مقادیر مجاز **interval** عبارت‌اند از **raw**، **1min**، **5min**، **15min**، **30min**، **60min**، **1h**. حالت **raw** هر مشاهده provider را به صورت یک کندل تک‌نقطه‌ای نگه می‌دارد؛ سایر حالت‌ها سطح شاخص را به OHLC تبدیل می‌کنند.

```
IndustryName, IndustryIndexCode, Timestamp, JalaliDate, Interval,
Open, High, Low, Close, Change, ChangePct
```

مصنوعی تولید نمی‌کند و فقط turnover یا volume حجم ندارد؛ تابع endpoint تهران است. این timezone دارای **Timestamp** را برمی‌گرداند provider آخرین روز موجود در.

رتبه‌بندی صنایع

```
ranking = att.rank_industries(
    metric="money_flow",
    top=10,
    ascending=False,
    include_client_type=True,
    include_orderbook=False,
    progress=False,
    refresh=False,
    max_workers=6,
)
```

های پشتیبانی‌شده alias و **metric**

معیار canonical	های رایج alias
IndexChangePct	change_pct , return
EqualWeightReturn	equal_weight_return
MedianReturn	median_return
AdvancePct	advance_pct , breadth
TotalValue	total_value , turnover
EstimatedNetIndividualValue	estimated_net_individual_value , money_flow
IndividualPower	individual_power , buyer_power

ردیف اول را نگه می‌دارد N همه ردیف‌های دارای معیار معتبر را برمی‌گرداند؛ مقدار مثبت فقط `top=None` دارد و معیار نهایی در `Rank` را همراه snapshot رتبه بزرگ‌تر به کوچک‌تر است. خروجی تمام ستون‌های ثبت می‌شود `attrs['ranking_metric']`.

cache، پوشش و محدودیت داده

عضویت کامل شاخص و تاریخچه کوتاه اعضا در یک payload مشترک می‌آیند. TTL پیش‌فرض cache حافظه یک ساعت است:

```
att.settings.industry_membership_cache_ttl = 3600.0
```

مقدار cache 0 را غیرفعال می‌کند. برای دریافت اجباری عضویت تازه از `refresh=True` استفاده کنید. cache فقط در حافظه همان process است و فایلی روی دیسک نمی‌نویسد.

محدودیت‌های رسمی این نسخه:

- وزن رسمی هر عضو، ضریب سهام شناور و divisor شاخص در منبع فعلی ارائه نمی‌شود؛ contribution دقیق نماد به واحد شاخص محاسبه نمی‌شود.
- تاریخچه تغییر اعضای شاخص وجود ندارد؛ تاریخچه اعضا universe امروز را روی روزهای قبل اعمال می‌کند.
- صنایع می‌توانند هم‌پوشانی داشته باشند و بعضی کدهای صنعت ممکن است در یک روز بدون عضو باشند.
- برآوردند و نام آن‌ها عمداً `EstimatedNetIndividualValue` و `EstimatedNetIndividualFlow` ، `EstimatedMarketCap` این موضوع را نشان می‌دهد.
- برای تصمیم معاملات، `TradeDate` ، `OrderBookCoverage` ، `ClientCoverage` ، `IsStale` ، `IsRealtimeFresh` و `attrs` را بررسی کنید.

تاریخچه محلی SQLite

ذخیره‌سازی کاملاً opt-in است؛ بدون path صریح هیچ فایلی نوشته نمی‌شود.

snapshotهای بازار

```
db = "market-history.sqlite"

snapshot_id = att.save_market_snapshot(db)
history = att.load_market_snapshots(db, symbol="فملی", limit=500)

# ذخیره شده live معنایی برای همان نمای alias
live_history = att.get_live_market_history(db, symbol="فملی", limit=500)
breadth_history = att.get_market_breadth_history(db, limit=100)
sector_history = att.get_sector_flow_history(db, limit=100)
overview_history = att.get_market_snapshot_summary_history(db, limit=100)
```

خروجی نماینده:

AsOf	InsCode	Symbol	Last Close	Volume	NoBackfill	SnapshotAtomic
2026-08-25 10:11:12+03:30	...	812200	73950	74200	فملی	True

و `CoverageStart`، `CoverageEnd`، `SnapshotFrameAttrs`، `NoBackfill`، `SnapshotAtomic` و `SourceAtomic` هایمانند `attrs` را توضیح می‌دهند `archive` محدود و کیفیت `SQLite`، `application-id`، `schema version`، `transaction` و کنترل دیتابیس بیگانه/خراب است.

record_to برای replay eventها

```
watcher = att.MarketWatcher(
    interval=2,
    max_updates=3,
    record_to=db,
    checkpoint_interval=100,
    record_max_records=10_000,
    record_retention_seconds=7 * 24 * 3600,
    storage_error_policy="raise",
)
for _ in watcher:
    pass

events = att.get_market_event_history(db, kind="delta", limit=1000)
```

اولین `event` هر `session` یک `checkpoint` کامل است؛ `delta`ها پس از آن `replay` می‌شوند و `pruning` فقط از مرز `checkpoint` معتبر انجام می‌شود. `storage_error_policy="raise"` انتخاب امن پیش فرض است. `record_market_event()` برای ثبت مستقیم `MarketEvent` موجود است.

archive_to برای رکوردهای مستقل

دو مسیر مستقل اند `record_to` و `archive_to`

```
att.get_market_overview(flow=0, archive_to=db)
att.get_market_messages(flow=0, top=20, archive_to=db)
att.get_instrument_state_changes(top=20, archive_to=db)

overview = att.get_market_overview_history(db, flow=0, limit=100)
messages = att.get_market_messages_history(db, flow=0, limit=100)
states = att.get_instrument_state_changes_history(db, symbol="فملی", limit=100)
```

های پشتیبانی شده است kind سطح پایین برای API `archive_market_records(path, kind, frame, source=...)` عملیات SQL در pagination قبل از `start/end`, `limit/offset` هستند؛ archive بخشی از هویت `source` و `selector` می شوند.

توابع مهم نگهداری:

```
info = att.check_market_history(db)
att.record_market_event(db, event, session_id="strategy-a")
```

عمومی اند migration/inspection برای `MARKET_HISTORY_SCHEMA_VERSION` و `MARKET_HISTORY_APPLICATION_ID`

فاندامنتال بازار، صندوق و تعدیل قیمت

EPS و P/E زنده

```
fundamentals = att.get_market_fundamentals(
    symbols=["فملی", "شتران"],
    pe_min=None,
    pe_max=None,
    positive_pe=False, # نگه دار audit را هم برای unavailable ردیف
    strict=False,
    allow_stale=False,
    archive_to="market-history.sqlite",
    max_requests=1,
    progress=False,
)
```

این تابع یک bulk snapshot می گیرد و EPS/P/E را از همان مشاهده می سازد؛ برای هر نماد سراغ منبع دیگری نمی رود.
schema ثابت:

```
InsCode, Symbol, Name, SectorCode, InstrumentType, Close, Last,
EPS, PE, PECalculated, EPSSource, PriceSource, PESTatus,
TradeDate, ExchangeTime, AsOf, SnapshotAgeSeconds,
IsRealtimeFresh, IsStale, Source, NoLookahead
```

خروجی نماینده:

Symbol	Close	EPS	PE	PECalculated	EPSSource	PriceSource	PEStatus	NoLookahead
10.0	7395	73950	فملی	True	market_watch	Close	ok	True
42100	شتران	<NA>	<NA>	True	missing_in_market_watch	Close	eps_missing	True

EPS صفر/خالی به مقدار جعلی تبدیل نمی‌شود. `PEStatus` یکی از `ok` , `price_missing` , `price_nonfinite` , `price_nonpositive` , `eps_missing` , `eps_nonfinite` , `eps_nonpositive` است؛ بودن در stale `IsStale` جداسازی snapshot. مفقود را به خطا تبدیل می‌کند `strict=True` selector مثبت را نگه می‌دارد و P/E فقط `positive_pe=True` زنده `attrs` می‌شود `attrs['stale_rejected']=True` نیست: خروجی تهی و exception `allow_stale=False` با stale دقیقاً شامل

`request_count,max_requests,missing_selectors,strict,stale_rejected,source,price_source,eps_source,no_loo`
`oahead,no_backfill,archive_path` است.

تاریخچه fundamentals

```
history = att.get_market_fundamentals_history(
    "market-history.sqlite",
    start="1403-05-01",
    end="1403-05-07",
    symbols="فملی",
    limit=1000,
)
print(history.attrs["no_lookahead"], history.attrs["current_eps_used"])
```

AsOf	Symbol	Close	EPS	PE	EPSSource	Source	NoLookahead
2026-08-24 10:00:00+03:30	10.0	7350	73500	فملی	market_watch	local_market_snapshot	True
2026-08-25 10:00:00+03:30	10.0	7420	74200	فملی	market_watch	local_market_snapshot	True

این `history` فقط از snapshot های ذخیره شده شما ساخته می‌شود: `attrs['no_backfill']=True` و `attrs['current_eps_used']=False` دیگر `missing_selectors,strict,source,price_source,eps_source,no_loo`
`ahead,coverage_start,coverage_end` هستند. EPS فعلی برای گذشته forward-fill نمی‌شود.

صندوق های قابل معامله

```
listed = att.list_listed_funds(progress=False)
# معادل صریح:
listed2 = att.list_funds(listed_only=True, progress=False)
```

schema: `list_listed_funds()` یک bulk call و بازار دارد و fuzzy join

InsCode, ISIN, Symbol, Name, Last, Close, Yesterday, Volume, Value, TradeCount, Low, High, NAV, NAV_Discount, Change, ChangePct, MarketCode

Symbol	InsCode	ISIN	Last	Close	NAV	NAV_Discount	Volume
افران	...	IRO3AFRZ0001	21650	21620	...	...	1250040

بدون `list_funds()` قرارداد هویتی را روشن می‌کنند `registry_joined=False` و `no_fuzzy_join=True` های `attrs` صندوق‌هاست و ستون/منبع متفاوتی دارد registry قدیمی API همان `listed_only`

رخدادهای تعدیل قیمت

```
adjustments = att.get_price_adjustments(
    "فملی", start="1402-01-01", end="1403-12-29", progress=False
)
latest = att.get_latest_price_adjustment("فملی", progress=False)
```

typed ثابت و schema

InsCode, Symbol, GregorianCalendar, JalaliDate, AdjustedClosingPrice, UnadjustedClosingPrice, AdjustmentAmount, CorporateTypeCode, CorporateActionType, IsConfirmedDPS, IdentityVerified, Source, FetchedAt

خروجی نماینده:

GregorianCalendar	JalaliDate	AdjustedClosingPrice	UnadjustedClosingPrice	AdjustmentAmount	CorporateTypeCode
CorporateActionType	IsConfirmedDPS	IdentityVerified			
2024-01-01	1402-10-11	8000	10000	2000	7
<NA>	False	True			

قرارداد مهم:

- فقط اختلاف ریاضی قیمت‌هاست `AdjustmentAmount = UnadjustedClosingPrice - AdjustedClosingPrice`
- provider خام `CorporateTypeCode` است؛ nullable `CorporateActionType` و `False` همیشه `IsConfirmedDPS` بدون تفسیر نگه داشته می‌شود.
- ثبت می‌شود `attrs['dps_reason']` و دلیل در `attrs['dps_available'] == False`
- رکورد با `InsCode` متفاوت رد می‌شود؛ نبود `InsCode` در خود رکورد با `IdentityVerified=False` و `InsCode` حل شده گزارش می‌شود.
- می‌دهد schema/dtypes/attrs صفر یا یک‌رديفی با همان `DataFrame` `get_latest_price_adjustment()`

```
assert adjustments["IsConfirmedDPS"].eq(False).all()
assert adjustments.attrs["dps_available"] is False
```

از این API برای ساخت «سری DPS» استفاده نکنید. TSETMC در این endpoint طبقه‌بندی قطعی سود نقدی ارائه نمی‌دهد.

اخزا و درآمد ثابت

قاعدهٔ تاریخ در نماد اخزا

شش رقم انتهایی نماد به صورت تاریخ شمسی **YYMMDD** تفسیر می‌شود؛ دو رقم سال با **13** یا **14** تکمیل می‌شود:

```
att.parse_treasury_maturity("020322|اخزا")
```

```
{
  'maturity_jalali': '1402/03/22',
  'maturity_gregorian': datetime.date(2023, 6, 12),
  'maturity_source': 'user_confirmed_symbol_jalali_ymdd'
}
```

```
att.parse_treasury_maturity("991117|اخزا")
```

```
{
  'maturity_jalali': '1399/11/17',
  'maturity_gregorian': datetime.date(2021, 2, 5),
  'maturity_source': 'user_confirmed_symbol_jalali_ymdd'
}
```

برای نماد non-match یا تاریخ شمسی نامعتبر، تابع exception نمی‌دهد و **None** برمی‌گرداند. در صورت تعارض یا نماد غیرقابل تفسیر، **maturity_date** یا **maturity_map** صریح بدهید؛ provenance ستون **MaturitySource** را بررسی کنید.

محاسبات deterministic

```
result = att.treasury_yield(
    price=800_000,
    maturity_date="2028-01-01",
    settlement_date="2026-01-01",
    face_value=1_000_000,
)
```

خروجی واقعی این مثال:

```
EffectiveAnnualYield 0.1180339887
SimpleAnnualYield    0.125
ContinuousYield      0.1115717757
BankDiscountYield    0.0986301370
MacaulayDuration     2.0
ModifiedDuration      1.7888543820
Convexity             4.8
DV01                  143.10835056
DiscountFactor        0.8
Status                ok
```

برای اوراق کوپنی:

```
cashflows = [
    ("2027-01-01", 80_000),
    ("2028-01-01", 80_000),
    ("2029-01-01", 1_080_000),
]
price = att.bond_price(0.08, cashflows, settlement_date="2026-01-01")
ytm = att.yield_to_maturity(price, cashflows, settlement_date="2026-01-01")
risk = att.bond_analytics(price, cashflows, settlement_date="2026-01-01")
```

`bond_price`، `day_count_fraction()` پشتیبانی می‌کند `ACT/365F`، `ACT/360`، `30/360` از قراردادهایی مانند `yield_to_maturity` و `bond_analytics` `compounding/frequency`، `clean/dirty price` و `accrued interest` پارامترهای `sign-changing` های مبهم یا `cashflow` دارند؛

اخزای زنده

```
ytm = att.get_treasury_yields(
    symbol=None,
    settlement_date=None,
    face_value=1_000_000,
    include_stale=False,
    min_volume=1,
    price_source="auto", # auto/bid/ask/last/close
    strict=False,
    source="tsetmc",     # مرجع join برای hybrid یا IFB
)
```

ستون‌های اصلی:

```
InsCode, ISIN, Symbol, MaturityJalali, Maturity, MaturitySource,
MaturityConflict, SettlementDate, DaysToMaturity, Tenor,
Price, PriceSource, NoTrade, InstrumentPriceAsOf, IsInstrumentStale,
BidPrice, AskPrice, DiscountFactor,
EffectiveAnnualYield, ContinuousYield, SimpleAnnualYield, BankDiscountYield,
MacaulayDuration, ModifiedDuration, Convexity, DV01,
Volume, Value, TradeCount, FaceValue, FaceValueSource,
DayCount, IsStale, SnapshotAgeSeconds, FetchedAt, Status
```

مقایسه `https://ifb.ir/ytm.aspx` را با جدول مرجع مجاز فرابورس ایران در TSETMC داده معاملاتی `source="hybrid"` گزارش می‌شود با provenance با bps جای قیمت ابزار را نمی‌گیرد و اختلاف در IFB می‌کند؛

```
ifb = att.get_ifb_yield_table(category="treasury")
print(ifb.attrs["reference_url"])
```

schema IFB:

```
Symbol, Price, LastTradeJalali, LastTradeDate, PublishJalali, PublishDate,
MaturityJalali, Maturity, Volume, ReferenceYTM,
ReferenceSimpleYield, ReferenceSource
```

تاریخچه YTM

```
history = att.get_treasury_yield_history(
    "اخزا090101",
    limit=20,
    include_today=True,
    price_source="close",
    max_requests=25,
    progress=False,
)
```

عینی همین تابع است. هر ردیف از قیمت همان روز ساخته می‌شود و `get_treasury_yields_history` alias است. `OHLC` مناسب نیست. ستون‌های تاریخچه شامل YTM است؛ قیمت تعدیل‌شده برای `auto_adjust=False`, `Close`, `Final`, `price_source` از همان `include_today` سررسید/قیمت است provenance و `IsPartial`, `FetchedAt`, `Status`، بالا `analytics` استفاده می‌کند live متناظر `source`.

منحنی بازده

```
curve = att.get_yield_curve(
    min_nodes=3,
    interpolation="log_discount",
    duplicate_policy="volume_weighted",
    extrapolate=False,
)

df_18m = curve.discount_factor(1.5)
zero_18m = curve.zero_rate(1.5)
fwd = curve.forward_rate(1.0, 2.0)
```

`YieldCurve` شامل `settlement_date` , `maturities` , `times` , `discount_factors` , `continuous_zero_rates` , `node_metadata` و `diagnostics` است. `extrapolate=False` را می‌گیرد.

ساخت مستقیم:

```
nodes = [
    {"Maturity": "2027-01-01", "DiscountFactor": 0.90},
    {"Maturity": "2028-01-01", "DiscountFactor": 0.80},
    {"Maturity": "2029-01-01", "DiscountFactor": 0.70},
]
curve = att.build_yield_curve(nodes, settlement_date="2026-01-01")
```

و `curve.diagnostics` دقیقاً کلیدهای `input_node_count` , `node_count` , `duplicate_count` , `duplicate_policy` و `monotonic_discount_enforced` discount factor نزولی بودن `guard` و `dedupe`، ورودی `audit` برای `metadata` را دارد؛ این است.

تاریخچه منحنی:

```
curves = att.get_yield_curve_history(
    limit=10,
    min_nodes=3,
    include_today=True,
    max_requests=50,
    progress=False,
)
```

هر `node` دارای `CurveID` , `CurveStatus` , `CurveNodeCount` , `CurveInterpolation` و `flag`های `CurvePricesNoLookahead` , `CurveUniverseNoLookahead` , `CurveNoLookahead` است. اگر `universe` تاریخی از `catalog` امروز بازیابی شود، `survivor-bias` در `diagnostics` صریح است و `CurveUniverseNoLookahead=False` می‌شود.

اختیار معامله

تحلیل‌های قیمت‌گذاری این بخش برای اختیار اروپایی هستند. مشخصات واقعی اعمال قرارداد، تعدیلات شرکتی و dividend yield از TSETMC حدس زده نمی‌شود.

ریاضی Black-Scholes

```
call = att.black_scholes_price(
    spot=100, strike=100, time_to_expiry=1,
    rate=0.05, volatility=0.20,
    option_type="call", dividend_yield=0.0,
)
greeks = att.black_scholes_greeks(100, 100, 1, 0.05, 0.20, "call")
bounds = att.option_price_bounds(100, 100, 1, 0.05, "call")
iv = att.implied_volatility(10.4505835722, 100, 100, 1, 0.05, "call")
```

خروجی واقعی و deterministic:

```
call = 10.4505835722

greeks = {
    'Delta': 0.6368306512,
    'Gamma': 0.0187620173,
    'Vega': 37.5240346917,
    'Vega1Pct': 0.3752403469,
    'ThetaPerYear': -6.4140275464,
    'ThetaPerDay': -0.0175726782,
    'Rho': 53.2324815454,
    'Rho100bp': 0.5323248155,
    'Status': 'ok'
}

bounds = (4.8770575499, 100.0)
iv = {'ImpliedVolatility': 0.1999999955, 'Status': 'ok', 'Iterations': 27}
```

واحدها مهم‌اند: **Vega** تغییر قیمت برای یک واحد volatility و **Vega1Pct** برای یک واحد درصد است؛ **ThetaPerYear/Day** و **Rho/Rho100bp** جدا گزارش می‌شوند. خروجی Greeks همیشه کلید **Status** دارد. **implied_volatility()** همواره dict می‌دهد و کلیدهای پایه آن **ImpliedVolatility**, **Status**, **Iterations** هستند؛ یکی از **missing**, **expiry**, **ok**, **non_converged**, **no_bracket**, **out_of_bounds** است. **out-of-bounds/no-bracket** ممکن است **LowerBound/UpperBound** و **non-converged** مقدار تشخیصی **CandidateVolatility** داشته باشد؛ عدم همگرایی exception نیست. فقط constraint‌های ورودی مانند **bracket/tolerance/style** نامعتبر **ValueError** می‌دهند.

snapshot اتمیک بازار اختیار

```
options = att.get_option_market(
    exchange=0,
    underlying="خودرو",
    max_requests=1,
    progress=False,
)
```

هر جفت call/put فقط وقتی metadata کافی و هویت سازگار داشته باشد وارد snapshot می‌شود. ستون‌های اصلی:

```
InsCode, PairID, PairSequence, ISIN, Symbol, Name, OptionType,
UnderlyingInsCode, UnderlyingSymbol, ContractSize, Strike,
BeginDate, EndDate, DaysToExpiry,
Last, Close, Yesterday, Volume, Value, TradeCount, NotionalValue,
OpenInterest, YesterdayOpenInterest, BidPrice, AskPrice, BidVolume, AskVolume,
UnderlyingLast, UnderlyingClose, Price, PriceSource,
AsOf, AsOfSource, SnapshotFreshnessKnown, PriceFreshnessKnown,
Stale, NoTrade, AnalyticsEligible, AnalyticsEligibilityReason,
MetadataConflict, Source
```

attrs های `atomic_snapshot` , `duplicate_pairs_dropped` , `incomplete_pairs_quarantined` ,
`exchange_event_freshness_known` و `malformed_pair_status` را توضیح می‌دهند snapshot کیفیت

`get_options_chain(underlying, fetch_oi=False)` فهرست قراردادها را می‌دهد و `list_options(underlying=None)` قدیمی API یک `dict` با کلیدهای دقیق `calls` , `puts` , `underlying_name` , `underlying_price` , `expiry_dates` پیشنهاد می‌شود `get_option_market()` حرفه‌ای analytics وجود ندارد. برای `price` برمی‌گرداند؛ کلید `market_time` و

IV، Greeks، parity و نقدشوندگی

```
analysis = att.analyze_option_chain(
    options=options,
    spot=None, # اگر معتبر باشد snapshot از
    risk_free_rate=0.30, # یا yield_curve=curve
    dividend_yield=0.0,
    valuation_date=None,
    exercise_style="european",
    parity_tolerance=None,
    allow_unverified_freshness=True,
)
```

ستون‌های تحلیلی افزوده‌شده:

```

TimeToExpiry, Spot, SpotSource, RiskFreeRate, RiskFreeRateSource,
DividendYield, DividendYieldSource,
ImpliedVolatility, ImpliedVolatilityBid, ImpliedVolatilityMid, ImpliedVolatilityAsk,
IVStatus, IVStatusBid, IVStatusMid, IVStatusAsk,
Delta, Gamma, Vega, Vega1Pct, ThetaPerYear, ThetaPerDay, Rho, Rho100bp,
PremiumContract, DeltaContract, GammaContract, VegaContract,
Vega1PctContract, ThetaPerYearContract, ThetaPerDayContract,
RhoContract, Rho100bpContract,
SpreadAbs, SpreadPct, QuotedDepth, LiquidityScore,
ParityResidual, ParityToleranceBand, ParityStatus, ImpliedForward,
GreeksStatus, AnalyticsReliability, AnalyticsWarning, AnalyticsComputed

```

خروجی نماینده:

```

Symbol OptionType Strike Price ImpliedVolatility Delta Vega1PctContract SpreadPct LiquidityScore
ParityStatus AnalyticsReliability
ضخود... call      3000  420.0          0.41  0.62          18320.0    0.03          0.81
ok verified_inputs
طخود... put      3000  265.0          0.39 -0.38          17790.0    0.04          0.76
ok verified_inputs

```

است و توصیهٔ آربیتراژ نیست؛ محدودیت وجه تضمین، سبک اعمال، کارمزد و امکان معامله را diagnostic فقط **ParityStatus** قابل اتکا ساخته نمی‌شود analytics معتبر نداشته باشید curve ندهید و **risk_free_rate** مدل نمی‌کند. اگر فقط زمانی ثبت می‌شود که کاربر مقدار را تعیین کرده باشد؛ مقدار صفر پیش‌فرض **DividendYieldSource='user_supplied'** نیست DPS به معنی کشف

PCR

```
pcr = att.option_put_call_ratios(analysis, group_by="underlying")
```

```

UnderlyingInsCode CallVolume PutVolume PCRVolum PCRVolumStatus CallValue PutValue PCRValue
PCRValueStatus CallOpenInterest PutOpenInterest PCROpenInterest PCROpenInterestStatus
65883838195688438 20000 13000 0.65 ok 9.2e9 5.1e9 0.55 ok
40000 20000 0.50 ok

```

است. خروجی برای هر معیار علاوه بر **underlying_expiry** یا **expiry**، **underlying**، **market** دقیقاً یکی از **group_by** را می‌دهد؛ نسبت با **PCROpenInterestStatus** و **PCRValueStatus** متناظر **status**، نسبت **zero_denominator** برابر **status** و **denominator** صفر

تاریخچه اختیار و snapshot محلی

```
# opt-in قیمت قرارداد؛ ردیف امروز server-side تاریخچه
history = att.get_option_history(
    "ضخود...", limit=10,
    include_today=True, max_requests=3, progress=False,
)

# بازار اختیار فقط با درخواست صریح کاربر روی فایل نوشته میشود
path = "option-snapshots.json"
att.save_option_snapshot(path, options=options)
saved = att.load_option_snapshots(path)
```

schema history:

```
Timestamp, InsCode, Symbol, Open, High, Low, Close, Last,
Volume, Value, TradeCount, OpenInterest,
BidPrice, AskPrice, BidVolume, AskVolume,
UnderlyingLast, UnderlyingClose, ContractSize, Strike, EndDate,
Price, PriceSource, Source, AsOf, Stale, NoTrade, AnalyticsEligible
```

attrهایی مانند `prices_no_lookahead`، `underlying_prices_no_lookahead`، `rates_no_lookahead`،
 دارای snapshot بررسی کنید. فایل `backtest` را برای `source_coverage` و `curve_applied` و `valuation_date_source`
 است dedupe و `write` و `writer` قفل، `OPTION_SNAPSHOT_SCHEMA_VERSION`

سایر API های بازار

این بخش قابلیت های قدیمی را در همان مرجع واحد نگه می دارد. API های legacy برای backward compatibility در دسترس اند و در بسیاری از خطاهای قدیمی `None` /پیام کنسول می دهند؛ API های جدید بیشتر از `typed exception` استفاده می کنند.

intraday

```
ticks_or_candles = att.get_intraday(
    "فملى",
    interval="1min",      # tick, 1min, 5min, 15min, 30min, 1h, 4h, 12h
    start="1403-05-01",   # برای امروز start/end حذف
    end="1403-05-03",
    progress=False,
)
```

خروجی candle معمولاً `Open, High, Low, Close, Volume` با index زمانی است. `tick` snapshot خام معاملات/قیمت را می دهد. `interval` بزرگتر از داده پایه `resample` می شود؛ تعطیلی و وقفه بازار را در محاسبه لحاظ کنید. نام های canonical بازه `1m`، `60min`، `4hour`، `240m`، `12h` و `tick`، `1min`، `5min`، `15min`، `30min`، `1h`، `4h` هستند؛ alias های عددی/کوتاه مانند

720 , 12hour و نیز raw / ticks پشتیبانی می‌شوند. این API رفتار legacy دارد: interval نامعتبر یا تاریخی که validator قدیمی نامعتبر تشخیص دهد را روی کنسول اعلام می‌کند و None برمی‌گرداند، نه اینکه عمداً ValueError قراردادشده‌ای بدهد.

اطلاعات نماد و سهامدار

```
detail = att.get_detail("فملی")
info = att.get_info("فملی")
stats = att.get_stats("فملی")
shareholders = att.get_shareholders("فملی", include_id=True)
capital = att.get_capital_increase("فملی")
```

- می‌دهد id و ردیف value ستون ، key برابر index کلید-مقدار با DataFrame|None یک get_detail()
- می‌دهند key برابر index کلید-مقدار با DataFrame get_stats() و get_info()
- شناسه را include_id=True تاریخچه روز را می‌دهد؛ date سهامداران فعلی و با get_shareholders(date=None) اضافه می‌کند.
- تاریخچه تغییر تعداد سهام/سرمایه را می‌دهد get_capital_increase()
- این توابع ins_code و keyword-only asset_type را نیز می‌پذیرند.

معرفی شرکت؛ API قدیمی unsupported

قدیمی مانده‌اند. معرفی ناشر داده import/signature فقط برای حفظ stock_introduction() و get_introduction() یا شبکه خطای زیر را می‌دهد resolver این پکیج قرار دارد؛ تابع همیشه پیش از source boundary کدال است و خارج از

```
try:
    att.get_introduction("فملی")
except att.UnsupportedDataSourceError as exc:
    print(exc)
```

```
UnsupportedDataSourceError: get_introduction/stock_introduction requires Codal data, which is outside
algotik-tse's TSETMC market-data source boundary
```

برای اطلاعات TSETMC از get_info() و get_detail() استفاده کنید. helper افشای ناشر یا history آن در این پکیج وجود ندارد.

فهرست نمادها و ابزارها

```
symbols = att.get_symbols(
    bourse=True, farabourse=True, payeh=True,
    haghe_taqadom=False, sandogh=False, bonds=False, options=False,
    mortgage=False, commodity=False, energy=False,
    payeh_color=None, output="dataframe", progress=False,
)

etfs = att.list_etfs(progress=False)
bonds = att.list_bonds(progress=False)
funds = att.list_funds(fund_type="fixed_income", progress=False)
listed_funds = att.list_listed_funds(progress=False)
options = att.list_options(underlying="خودرو", progress=False)
indices = att.list_indices(progress=False)
members = att.get_index_companies("شاخص صنعت بانکها", progress=False)
industry_indices = att.list_industry_indices(progress=False)
industry_snapshot = att.get_industry_snapshot("بانک", progress=False)
```

بازار/نوع ابزار را نگه می‌دارد metadata `dataframe` فقط نام نمادها را می‌دهد؛ `get_symbols(output="list")` `payeh_color` را صریح تنظیم `asset-type flag`، اشتباه universe است. برای جلوگیری از `قرمز`، `نارنجی`، `زرد` یکی از `payeh_color` کنید.

اوراق و سررسید را فهرست می‌کند metadata `list_bonds()` را می‌دهد NAV/discount اطلاعات معامله و `list_etfs()` صندوق‌هاست؛ registry `list_funds()` بالاست fixed-income های API دقیق اخزا در analytics ولی دقیق می‌دهد InsCode/ISIN بازار را با feed فقط ابزارهای واقعاً قابل معامله در `list_listed_funds()`

شاخص‌ها

```
index_history = att.get_history("شاخص کل", limit=100, progress=False)
industry_history = att.get_industry_history("بانک", limit=100, progress=False)
members = att.get_industry_members("بانک", progress=False)
```

برای شاخص‌های عمومی همچنان `get_history()` را به‌کار ببرید. برای شاخص صنعت، API های اختصاصی فصل [شاخص‌ها و تحلیل صنایع](#) عضویت رسمی، تاریخچه اعضا، snapshot و intraday را یکدست ارائه می‌کنند. schema شاخص با سهام فرق دارد و volume جعلی ساخته نمی‌شود. `list_indices()` و `get_index_companies()` برای سازگاری با کد قدیمی حفظ شده‌اند.

ارز و سکه

این API legacy از TGJU استفاده می‌کند و برای backward compatibility حفظ شده است:

```
fx = att.get_currency(
    "dollar", start="1403-01-01",
    output_type="standard", date_format="jalali", progress=False,
)

coins = att.get_currency(["seke", "nim-seke"], limit=10, progress=False)
```

نام‌های انگلیسی دقیق:

```
dollar, euro, yuan, dirham, pound, lira,
dollar-sana-sell, dollar-sana-buy,
dollar-nima-buy, dollar-nima-sell,
dollar-sarafimelli-buy,
seke, seke-bahar-azadi, nim-seke, rob-seke, seke-gerami
```

نام‌های فارسی پشتیبانی‌شده شامل ربع، نیم سکه، سکه بهار آزادی، سکه، لیر، پوند، درهم، یوان، یورو، دلار سکه گرمی، سکه و صورت‌های سنا/نیم در settings است. spelling کلیدها را دقیق رعایت کنید؛ نام سکه در API انگلیسی seke است، نه sekke.

خروجی standard ستون‌های Open, High, Low, Close دارد. multi-currency یک DataFrame با MultiIndex ستونی می‌دهد. save_path در اینجا نیز پوشه است.

تنظیمات و خطاها

تنظیمات singleton:

```
from algotik_tse import settings

settings.ssl_verify = True          # پیش فرض و توصیه شده
settings.timeout = 10              # ثانیه
settings.max_retries = 3
settings.retry_backoff_factor = 0.3
settings.rate_limit_delay = 0.3    # فاصله حداقل شروع درخواست‌ها

settings.market_snapshot_freshness_seconds = 120.0
settings.market_clock_skew_tolerance_seconds = 5.0
settings.client_volume_consistency_tolerance = 0.05
settings.industry_membership_cache_ttl = 3600.0
settings.order_book_max_requests = 250
settings.trade_max_requests = 250
```

خراب CA فقط برای محیط کنترل‌شده با opt-out است. True پیش فرض TLS verification

```
from algotik_tse.http_client import safe_get

response = safe_get("https://cdn.tsetmc.com/...", verify=False)
```

این opt-out را سراسری نکنید. URL های TSETMC همگی HTTPS هستند.

قرارداد دقیق : `safe_get(url, **kwargs)`

- نامجاز با URL/Codal path های مجاز باشد؛ provider و داخل HTTP(S) باید `url: str` رد می شود I/O و session، rate-limit، پیش از `UnsupportedDataSourceError`
- فقط `headers=settings.headers`، `timeout=settings.timeout` و `verify=settings.ssl_verify` های default نداده باشد اعمال می شوند caller override وقتی
- هستند. نوع نادرست اولی/دومی wrapper پارامترهای خود `allow_redirects: bool=True` و `max_redirects: int=5` است `TypeError` و `ValueError`، `max_redirects<0`
- درخواست زیرین همیشه `allow_redirects=False` دارد. redirect فقط `scheme, host, effective-` same-origin (port) و hop-by-hop است؛ `UnsupportedDataSourceError` cross-origin و loop/عبور از سقف `requests.exceptions.TooManyRedirects` می دهد.
- redirect در caller های header دوباره فرستاده نمی شود؛ redirect فقط روی درخواست اول اعمال می شود و روی `params` حفظ می شوند same-origin
- خطاهای transport خود `requests` بعد از retry propagate می شوند. `safe_get` به تنهایی روی status 4xx/5xx `raise_for_status()` نمی کند؛ API مصرف کننده باید پیش از parse آن را به خطای typed خود تبدیل کند.

سلسله مراتب خطا:

```
AlgotikTSEError
├─ AmbiguousSymbolError
├─ ConnectionError
├─ DataParsingError
├─ InvalidParameterError
├─ StockNotFoundError
└─ UnsupportedDataSourceError
```

legacy سطح بالای پکیج نیست. توابع export برای سازگاری داخلی وجود دارد ولی exceptions در ماژول `RateLimitError` و پیام کنسول بدهند؛ قرارداد هر تابع را بررسی کنید `None`، exception ممکن است به جای

الگوی امن:

```
try:
    df = att.get_price_adjustments(ins_code="35425587644337450")
except att.InvalidParameterError as exc:
    print("bad input", exc)
except att.AmbiguousSymbolError as exc:
    print("pass ins_code", exc)
except att.ConnectionError as exc:
    print("provider unavailable", exc)
except att.DataParsingError as exc:
    print("provider schema changed", exc)
except att.UnsupportedDataSourceError as exc:
    print("outside supported sources", exc)
```

مرجع تفصیلی همه توابع

در این بخش همه export های عمومی پوشش داده شده‌اند. هر ردیف API در کنار همان تابع، ورودی‌ها و گزینه‌های اختصاصی، خروجی، خطاهای اصلی و مثال را توضیح می‌دهد. جدول‌های پارامتر مشترک فقط تعریف اصطلاحات را یکسان نگه می‌دارند؛ signature دقیق هر تابع نیز بلافاصله پیش از جدول همان گروه آمده است.

نمای کلی export ها و نام‌های قدیمی

جدول زیر inventory کامل export های `algotik_tse.__all__` است. جزئیات خروجی در بخش موضوعی مربوط آمده است.

هویت، تنظیمات و خطا

کاربرد	Export
تنظیمات singleton شبکه/بازار	<code>settings</code>
هویت immutable ابزار	<code>InstrumentRef</code>
نرمال‌سازی و حل دقیق هویت	<code>normalize_instrument_text</code> , <code>validate_ins_code</code> , <code>resolve_instrument</code>
خطاهای عمومی	<code>AlgotikTSEError</code> , <code>AmbiguousSymbolError</code> , <code>ConnectionError</code> , <code>DataParsingError</code> , <code>InvalidParameterError</code> , <code>StockNotFoundError</code> , <code>UnsupportedDataSourceError</code>

قیمت، trades، client type و اطلاعات نماد

Alias/legacy عمومی	Export canonical
<code>stock</code>	<code>get_history</code>
<code>stock_RI</code> , <code>stock_RL</code>	<code>get_client_type</code>
<code>stock_capital_increase</code>	<code>get_capital_increase</code>

Alias/legacy عمومی	Export canonical
stock_intraday	get_intraday
—	get_trades , get_live_trades
stockdetail	get_detail
stock_information	get_info
stock_statistics	get_stats
stock_introduction (همیشه unsupported)	get_introduction (همیشه unsupported)
shareholders	get_shareholders
stocklist	get_symbols
currency_coin	get_currency

پارامترهای قدیمی نیز پذیرفته می‌شوند: `stock` → `symbol` ، `values` → `limit` ، `tse_format` → `raw` ، `multi_stock_drop` / `multi_currencies_drop` → `dropna` و `output_type="complete"` → `"full"` در مسیرهای مربوط. برای کد جدید نام canonical را به کار ببرید.

live، سفارش، watcher و history محلی

ها Export
get_market_snapshot , get_market_client_type , get_order_book , get_live_market , get_live_symbol
get_order_book_history , get_orderbook_history , get_queue , get_queue_history
MarketEvent , MarketWatcher , watch_market
get_market_messages , get_instrument_state_changes , get_market_overview , get_market_breadth , get_sector_flow
MARKET_HISTORY_SCHEMA_VERSION , MARKET_HISTORY_APPLICATION_ID , check_market_history
save_market_snapshot , load_market_snapshots , get_live_market_history
get_market_overview_history , get_market_snapshot_summary_history , get_market_breadth_history , get_sector_flow_history
record_market_event , get_market_event_history , archive_market_records
get_market_messages_history , get_instrument_state_changes_history

سه wrapper صفرآرگومان legacy نیز عمومی‌اند: `market_watch()` ، `market_client_type()` ، `market_data()` . `market_data()` wrapper deprecated است؛ برای کد جدید `get_market_snapshot()` یا `get_live_market()` را انتخاب کنید.

fundamentals، تعديل قيمت و ابزارها

ها Export

`get_market_fundamentals` , `get_market_fundamentals_history`

`get_price_adjustments` , `get_latest_price_adjustment`

`list_options` , `get_options_chain` , `list_etfs` , `list_bonds` , `list_funds` , `list_listed_funds`

`list_indices` , `get_index_companies`

`list_industry_indices` , `get_industry_members` , `get_industry_snapshot`

`get_industry_history` , `get_industry_members_history` , `get_industry_intraday` , `rank_industries`

درآمد ثابت

ها Export

`IRAN_TREASURY_FACE_VALUE` , `YieldCurve`

`parse_treasury_maturity` , `day_count_fraction` , `treasury_yield`

`bond_price` , `yield_to_maturity` , `bond_analytics` , `build_yield_curve`

`get_ifb_yield_table` , `get_treasury_yields`

`get_treasury_yield_history` , `get_treasury_yields_history`

`get_yield_curve` , `get_yield_curve_history`

اختيار معامله

ها Export

`OPTION_SNAPSHOT_SCHEMA_VERSION`

`black_scholes_price` , `black_scholes_greeks` , `option_price_bounds` , `implied_volatility`

`get_option_market` , `analyze_option_chain` , `option_put_call_ratios`

`get_option_history` , `save_option_snapshot` , `load_option_snapshots`

signature های کاربرد

```
get_live_market(symbol=None, *, strict=False)
get_live_symbol(symbol=None, *, ins_code=None, fallback="none")
get_order_book(symbol=None, *, selector_strict=False)
get_queue(symbol=None, side="both", strict=True, *, selector_strict=False)
get_trades(symbol=None, *, ins_code=None, start=None, end=None,
            include_canceled=False, max_requests=None, raw=False, progress=True)
get_market_messages(flow=0, top=20, since_id=None, *, archive_to=None)
get_instrument_state_changes(top=20, since_id=None, *, archive_to=None)
get_market_overview(flow=0, *, archive_to=None)
```

پارامترهایی که با `_` شروع می‌شوند seam داخلی تست‌اند و API کاربر محسوب نمی‌شوند، حتی اگر در `inspect.signature` دیده شوند.

روش خواندن مرجع تفصیلی

هر signature زیر عین خروجی پایدارشده `inspect.signature` است؛ فقط آدرس حافظه `safe_get` به خود نام `safe_get` نرمال شده است. نوع‌هایی که در signature قدیمی annotation ندارند در جدول پارامترها مشخص شده‌اند. مقدار `None` معمولاً یعنی «فیلتر/override اعمال نشود»، نه رشته `"None"`. پارامترهای مشترک فقط یک‌بار در واژه‌نامه زیر توضیح داده می‌شوند و هر مدخل API علاوه بر آن، `override` و `constraint` خاص خود را می‌گوید. «خطاها» خطاهای اصلی قرارداد است، نه فهرست همه خطاهای ممکن Python/pandas.

پارامترهای مشترک هویت، تاریخچه و خروجی

نام	معمول default و type	معنا و constraint
<code>symbol</code> / <code>symbols</code>	<code>str Iterable[str]</code> <code>''</code> API <code>None</code> ؛ بسته به <code>None</code> یا	نماد فارسی، <code>InsCode</code> یا مجموعه آن‌ها؛ برای هویت مبهم <code>ins_code</code> بدهید. <code>symbols=None</code> یعنی کل <code>universe</code> .
<code>ins_code</code> / <code>inscode</code>	<code>str int None = None</code>	شناسه دقیق ۱ تا ۲۰ رقم ASCII؛ با <code>selector</code> متناقض مجاز نیست. فقط <code>inscode</code> spelling تاریخی <code>reader</code> وضعیت است.
<code>asset_type</code>	<code>str = 'auto'</code>	تعیین <code>provider</code> نوع را از <code>auto</code> حل هویت؛ <code>hint</code> می‌کند.
<code>start</code> , <code>end</code> , <code>date</code>	<code>str date datetime None</code>	مرز شامل ابتدا/انتها؛ ISO شمسی یا میلادی در API های بازار. <code>date</code> سهامداران <code>None</code> یعنی آخرین مشاهده.
<code>limit</code>	ها <code>reader</code> یا <code>int = 0</code> <code>1000</code>	۰ در <code>history</code> های <code>provider</code> یعنی بدون محدودیت بعد از فیلتر؛ در <code>SQLite</code> حداکثر صفحه. منفی نامعتبر است.
<code>offset</code>	<code>int = 0</code>	بعد از فیلترها؛ نامنفی <code>SQL offset</code> .

نام	معمول default و type	معنا و constraint
<code>include_today</code>	<code>bool = False</code>	ردیف زنده امروز؛ ممکن است در ساعات opt-in بازار آخرین مشاهده همین لحظه باشد و provenance دارد
<code>raw</code>	<code>bool = False</code>	ممکن order-book ؛ در provider نزدیک schema باشد delta/raw-mixed است
<code>output_type</code>	<code>str = 'standard'</code>	خروجی؛ مقادیر دقیق تابعی اند schema (<code>standard</code> / <code>full</code> یا <code>long</code> / <code>wide</code>). است <code>full</code> قدیمی <code>complete</code> alias
<code>date_format</code>	<code>str = 'jalali'</code>	شکل index/ستون تاریخ؛ <code>jalali</code> , <code>gregorian</code> , <code>both</code> در مسیرهای پشتیبانی شده.
<code>ascending</code>	<code>bool = True</code>	ترتیب زمانی خروجی بعد از فیلتر.
<code>progress</code>	<code>bool = True</code>	فقط پیام پیشرفت؛ در داده و schema اثر ندارد.
<code>save_to_file</code>	<code>bool = False</code>	ذخیره CSV opt-in.
<code>save_path</code>	<code>str Path None</code>	پوشه مقصد، نه نام فایل؛ بدون <code>save_to_file=True</code> نوشته نمی شود.
<code>dropna</code>	<code>bool = True</code>	حذف ردیف/ستون کاملاً تهی طبق قرارداد همان API؛ ردیف partial معنادار order-book حفظ می شود.
<code>return_type</code>	<code>str None</code>	history سازگاری برای انتخاب نوع خروجی در alias را <code>output_type</code> قیمت/ارز؛ برای کد جدید ترجیح دهید
<code>max_requests</code>	<code>int None</code>	سقف سخت fan-out قبل/حین I/O؛ معنای دقیق شمارش در مدخل تابع آمده است.
<code>strict</code> / <code>selector_strict</code>	<code>bool</code>	را فعال match خطای داده/فیلتر بدون strict؛ scalar انتخاب <code>selector_strict</code> می کند؛ مبهم/گم شده را خطا می کند
<code>allow_stale</code> / <code>include_stale</code>	<code>bool</code>	اجازه نگاه داشتن snapshot/اوراق stale؛ stale بودن همچنان در ستون attrs گزارش می شود.
<code>archive_to</code> , <code>record_to</code> , <code>snapshot_path</code> , <code>path</code>	<code>str Path None</code>	فایل SQLite/JSON صریح؛ نوشتن فقط با opt-in. <code>path</code> در reader/writer اجباری است.
<code>kwargs</code>	های سازگاری keyword	فقط alias های مستند مانند <code>values</code> , <code>stock</code> , <code>tse_format</code> و نام های انگلیسی keyword ; <code>stocklist</code> ناشناخته قرارداد عمومی نیست.
<code>_request</code> , <code>_snapshot</code> , <code>_client_type</code> , <code>_clock</code> , <code>_wait</code> , <code>_random</code> , <code>_recorded_at</code> , <code>_live_fetch</code> , <code>_request_budget_state</code>	داخلی seam	فقط تزریق deterministic در تست؛ برای مصرف عادی استفاده نشود و BC عمومی برای آن تضمین نمی شود.

پارامترهای مشترک live، watcher و SQLite

نام	type/default	معنا و constraint
flow	int None = 0	بازار/جریان provider؛ None در analytics یعنی بدون فیلتر.
sector	str Iterable None	فیلتر SectorCode .
instrument_types	Iterable[int] None	. 300,303,309 ابزار؛ پیش‌فرض تحلیلی سهام universe
traded_only	bool = False	فقط ابزار دارای معامله را در denominator نگه می‌دارد.
include_base_market	bool = True	نوع 309 بازار پایه را در universe پیش‌فرض نگه می‌دارد.
side	str = 'both'	برای صف both یا buy , sell
complete_only	bool = False	فقط snapshotهای پنج‌سطح کامل.
top	int = 20	تعداد پیام/وضعیت در درخواست؛ مثبت و bounded.
since_id	int str None	فیلتر client-side رکوردهای پس از شناسه؛ cursor provider نیست.
source	str = 'tsetmc'	می‌تواند fixed-income؛ در archive هویت/provenance باشد hybrid یا حالت مستند tsetmc
session_id	str None	شناسه session watcher برای نوشتن/فیلتر replay.
kind	str None	kind یا initial/delta/heartbeat/resync های event پشتیبانی‌شده archive
recorded_at , as_of	timestamp-like یا None	زمان مشاهده؛ None یعنی ساعت تهران/UTC داخلی معتبر تابع.
max_records , record_max_records	int = 10000	بر اساس تعداد؛ مثبت retention
retention_seconds , record_retention_seconds	float None	یعنی غیرفعال None زمانی؛ retention
interval	str در intraday؛ float=1.0 در watcher	یا (tick/1min/5min/15min/30min/1h) candle interval بر حسب ثانیه polling فاصله
max_updates	int None	سقف eventهای iterator؛ None یعنی تا stop.
notifications	پیش‌فرض iterable، ('messages', 'state')	فقط این دو notification پشتیبانی می‌شوند؛ Codal وجود ندارد.
error_policy , callback_error_policy , storage_error_policy	str	enum های دقیق: error_policy∈{retry,raise,stop} ، callback_error_policy∈{raise,ignore,stop} و storage_error_policy∈{raise,ignore} ؛ مقدار نامعتبر خطا است I/O پیش از
max_backoff , jitter , request_timeout	float None	هر درخواست؛ نامنفی/مثبت طبق timeout و jitter ،سقف backoff کلاس.
max_consecutive_retries , max_retries	int None	سقف retry؛ max_retries نام قدیمی سازگار است.

نام	type/default	معنا و constraint
<code>include_initial</code> , <code>emit_heartbeats</code> , <code>copy_snapshot</code> , <code>record_heartbeats</code>	<code>bool</code>	کنترل emission/copy/persistence event ها.
<code>notification_top</code> , <code>max_seen_notifications</code> , <code>checkpoint_interval</code>	<code>int</code>	بین ۱ و ۱۰۰۰؛ دو مقدار دیگر مثبت <code>notification_top</code> . بین ۱ و ۱۰۰۰۰ است <code>record_max_records</code> .

پارامترهای درآمد ثابت و اختیار

نام	type/default	معنا و constraint
<code>settlement_date</code> , <code>maturity_date</code> , <code>issue_date</code> , <code>valuation_date</code>	<code>None</code> یا date-like	تاریخ تسویه/سررسید/انتشار/ارزش‌گذاری؛ <code>None</code> در live یعنی امروز تهران.
<code>face_value</code>	<code>float</code> ؛ اخزا <code>1_000_000.0</code>	ارزش اسمی مثبت؛ <code>IRAN_TREASURY_FACE_VALUE</code> همین default است.
<code>day_count</code> / <code>convention</code>	<code>str='ACT/365F'</code>	پشتیبانی‌شده؛ تاریخ پایان باید بعد از شروع convention باشد.
<code>price</code> , <code>annual_yield</code> , <code>coupon_rate</code> , <code>accrued_interest</code>	<code>float</code>	قیمت/بازده/کوپن/بهرهٔ تحقق‌یافته؛ bounds در تابع math اعتبارسنجی می‌شود.
<code>cashflows</code>	iterable (date, amount) یا <code>None</code>	جریان‌های نقدی؛ در حالت <code>None</code> از maturity/face/coupon ساخته می‌شود.
<code>compounding</code> , <code>frequency</code> , <code>price_type</code>	<code>str</code> , <code>int</code> , <code>str</code>	نوع مرکب، دفعات سالانه و <code>dirty/clean</code> .
<code>nodes</code>	DataFrame/iterable mapping	rate/discount و maturity شامل curve های node: تعیین‌کنندهٔ تکرار است duplicate policy.
<code>interpolation</code> , <code>extrapolate</code> , <code>duplicate_policy</code>	<code>log_discount</code> , تابعی , <code>False</code>	سیاست interpolation discount: extrapolation opt-in؛ مستند policy یا <code>error</code> duplicate.
<code>price_source</code>	<code>str='auto'</code>	انتخاب <code>Last/Close/Final/bid/ask</code> با provenance.
<code>maturity_map</code>	<code>None</code> یا mapping	جلوگیری fuzzy صریح نماد→سررسید؛ از حدس override می‌کند.
<code>spot</code> , <code>strike</code> , <code>option_price</code> , <code>volatility</code>	<code>float</code>	نامنفی volatility نامنفی و premium، مثبت spot/strike.
<code>time_to_expiry</code> , <code>rate</code> , <code>dividend_yield</code>	<code>float</code>	سال تا سررسید، نرخ بدون ریسک و yield پیوسته.
<code>option_type</code> , <code>exercise_style</code>	<code>str='call'</code> , <code>str='european'</code>	را می‌پذیرد European فعلی فقط math <code>call/put</code> :

نام	type/default	معنا و constraint
<code>lower_volatility</code> , <code>upper_volatility</code> , <code>tolerance</code> , <code>max_iterations</code>	<code>0.0</code> , <code>5.0</code> , تابعی، تابعی	و شمار <code>lower < upper</code> solver: bracket و همگرایی مثبت iteration.
<code>risk_free_rate</code> , <code>yield_curve</code>	<code>None</code> یا <code>scalar/curve</code>	نرخ ثابت یا <code>YieldCurve</code> ; اگر هر دو داده شوند قرارداد تحلیل آن‌ها را اعتبارسنجی می‌کند.
<code>parity_tolerance</code> , <code>liquidity_weights</code>	<code>float</code> <code>mapping</code> <code>None</code>	band parity scoring: <code>None</code> یعنی default و وزن‌های band parity داخلی مستند.

پارامترهای کم‌تکرار نیز بخشی از قرارداداند:

دامنه	پارامترها و معنا
قیمت history	متناظر volume تعدیل (<code>adjust_volume</code> (<code>bool=False</code>)) OHLC: تعدیل (<code>auto_adjust</code> (<code>bool=True</code>))
فهرست بازار	<code>bourse</code> , <code>farabourse</code> , <code>payeh</code> (<code>bool=True</code>) و <code>haghe_taqadom</code> , <code>sandogh</code> , <code>bonds</code> , <code>options</code> , <code>mortgage</code> , <code>commodity</code> , <code>energy</code> (<code>bool=False</code>) <code>inclusion</code> : <code>output</code> (<code>str='dataframe'</code>) <code>format</code> : <code>name</code> (<code>str list=''</code>) نام/InsCode <code>fund_type</code> (<code>str list None</code>) <code>category</code> : <code>index_name</code> (<code>str</code>) شناسه سهامدار (<code>include_id</code> (<code>bool=False</code>)) شاخص؛
fundamentals	مثبت معتبر P/E فقط (<code>positive_pe</code> (<code>bool=True</code>)) مرزهای شامل: (<code>pe_min</code> , <code>pe_max</code> (<code>float None</code>))
live/option	<code>category</code> کد بازار اختیاری: (<code>exchange</code> (<code>int=0</code>)) <code>point opt-in</code> : (<code>fallback</code> (<code>str='none'</code>)) <code>lock_timeout</code> (<code>float=10.0</code>) و <code>stale_lock_seconds</code> (<code>float=300.0</code>) <code>IFB</code> دسته (<code>str='treasury'</code>) قفل فایل مثبت.
fixed math	<code>bump_size</code> (<code>float=0.0001</code>) شوک DV01: (<code>rate_compounding</code> (<code>str='effective'</code>), <code>rate_frequency</code> (<code>int=1</code>); <code>enforce_monotonic_discount</code> (<code>bool=True</code>) <code>guard curve</code> .
storage	<code>event</code> (<code>MarketEvent</code>) رخداد ورودی و <code>frame</code> (<code>DataFrame</code>) batch archive.
resolver	الزام فعالیت (<code>require_active</code> (<code>bool=True</code>)) انتخاب ورودی: (<code>selector</code> (<code>Any</code>)).

فیلدهای باقی‌مانده `MarketEvent` همگی `constructor contract` هستند: (`sequence` (`int`) شماره افزایشی، `changed_order_levels` (`tuple[tuple[str,int],...]`) `delta`، و (`changed_incodes` (`tuple[str,...]`) `notification`، `market_state_changed` (`bool`)، `notification_tokens` (`tuple[str,str,str]`) `cursor` `state_changes` (`DataFrame|None`)، `cursor_before` / `cursor_after` (`int`)، `retry_count` (`int`)، `retry_error` (`persistence_status` / `persistence_error` (`str|None`) و (`str|None`))، `notification_errors` (`tuple[str,...]`) هستند. فیلد (`is_active` (`bool|None`) در `InstrumentRef` سه حالت فعال/غیرفعال/نامعلوم دارد.

نوع‌ها، ثابت‌ها، تنظیمات و exception‌های عمومی

```
InstrumentRef(ins_code: 'str', symbol: 'str | None' = None, name: 'str | None' = None, asset_type: 'str' = 'unknown', is_active: 'bool | None' = None, provenance: 'str' = '', selector: 'str | None' = None) -> None
```

نماد/نام، نوع دارایی، وضعیت فعال، منبع اثبات و canonical هویت است؛ فیله‌ها به ترتیب شناسهٔ `dataclass immutable` ورودی هستند. ساخت مستقیم فقط برای داده از قبل اعتبارسنجی شده مناسب است؛ مسیر عادی `selector` گم‌شده/اضافی است `field` برای `TypeError` استاندارد constructor است. خطای `resolve_instrument()`

```
MarketEvent(kind: 'str', sequence: 'int', fetched_at: 'pd.Timestamp', trade_date:
'Optional[_dt.date]', snapshot: 'dict[str, Any]', changed_incodes: 'tuple[str, ...]' = (),
changed_order_levels: 'tuple[tuple[str, int], ...]' = (), market_state_changed: 'bool' = False,
notification_tokens: 'tuple[str, str, str]' = ('', '', ''), messages: 'Optional[pd.DataFrame]' = None,
state_changes: 'Optional[pd.DataFrame]' = None, cursor_before: 'int' = 0, cursor_after: 'int' = 0,
retry_count: 'int' = 0, retry_error: 'Optional[str]' = None, notification_errors: 'tuple[str, ...]' =
(), persistence_status: 'Optional[str]' = None, persistence_error: 'Optional[str]' = None) -> None
```

هستند؛ `mutable` کپی دفاعی ولی `copy_snapshot=True` با `snapshot/messages/state_changes` است. `event watcher` را نگه می‌دارند. مثال ساخت دستی لازم نیست؛ نمونهٔ `persistence provenance` و رشته‌های خطا `delta/cursor` ها `tuple` `watch_market()` موضوعی بالاتر است

```
YieldCurve(settlement_date: datetime.date, maturities: tuple, times: tuple, discount_factors: tuple,
continuous_zero_rates: tuple, day_count: str = 'ACT/365F', interpolation: str = 'log_discount',
extrapolate: bool = False, node_metadata: tuple = <factory>, diagnostics: dict = <factory>) -> None
```

بسازید. `build_yield_curve()` است؛ آن را با `metadata/diagnostics` محاسباتی با آرایه‌های هم‌طول و `curve immutable` می‌دهند `ValueError`، `extrapolate=False` خارج از محدوده با `interpolation` متدهای

ثابت export	type/value	معنا
MARKET_HISTORY_SCHEMA_VERSION	int = 2	نسخهٔ SQLite market history schema.
MARKET_HISTORY_APPLICATION_ID	int = 1096045381	برای رد فایل نامرتبط SQLite application id
IRAN_TREASURY_FACE_VALUE	float = 1_000_000.0	ارزش اسمی پیش‌فرض اخزا، نه override اجباری همهٔ اوراق.
OPTION_SNAPSHOT_SCHEMA_VERSION	int = 1	نسخهٔ سند snapshot JSON اختیار؛ reader mismatch را رد می‌کند.

`ssl_verify: bool=True`، `timeout: int|float=10`، `max_retries: int=3`، `retry_backoff_factor: float=0.3`، `rate_limit_delay: float=0.3`، `market_snapshot_freshness_seconds: float=120.0`، `market_clock_skew_tolerance_seconds: float=5.0`، `client_volume_consistency_tolerance: float=0.05`، `industry_membership_cache_ttl: float=3600.0`، `order_book_max_requests: int=250`، `trade_max_requests: int=250` و `order_book_discovery_lookback_days: int=10` هستند. `headers: dict`، `mapping` و پایه `url_*: str` های روز/ارز/صندوق/بازار `provider` قرارداد تنظیم `source-boundary` می‌شود URL تغییر را نقض کند و برای مصرف عادی توصیه نمی‌شود

دارند `AlgotikTSEError` مشترک `base` و `(*args)` ارث‌بردهٔ constructor های عمومی همگی `exception` برای `DataParsingError`، `HTTP/provider` برای `ConnectionError`، برای چند هویت هم‌رتبه `AmbiguousSymbolError`، برای نبود هویت و `StockNotFound`، برای ورودی نامعتبر `InvalidParameterError`، نامن `schema/identity`

در فصل تنظیمات است catch برای خروج از مرز منبع. نمونه `UnsupportedDataSourceError`

قرارداد	constructor
base exception: <code>args: tuple[Any, ...]</code> را مانند <code>Exception</code> می‌دارد؛ <code>return</code> خودش ندارد.	<code>AlgotikTSEError</code>
چند هویت <code>exact</code> هم‌رتبه؛ راه‌حل ارائه <code>ins_code</code> است.	<code>AmbiguousSymbolError</code>
<code>HTTP/status/redirect/provider failure</code> با <code>cause</code> اصلی (<code>raise ... from exc</code>).	<code>ConnectionError</code>
کور معمولاً مناسب نیست <code>retry</code> هویت/فایل ناسازگار؛ <code>payload/schema</code> .	<code>DataParsingError</code>
<code>I/O</code> های جدید پیش از <code>API</code> ورودی؛ در <code>constraint</code> .	<code>InvalidParameterError</code>
انجام نمی‌شود <code>fuzzy-first-hit</code> پیدا نشده؛ <code>selector exact</code> .	<code>StockNotFoundError</code>
است <code>fail-before-I/O</code> نمونه <code>get_introduction</code> boundary: نیازمند منبع خارج <code>API</code> .	<code>UnsupportedDataSourceError</code>
نامعتبر <code>field/shape</code> و خطای <code>object curve</code> بالا؛ خروجی <code>constructor dataclass</code> <code>build_yield_curve</code> ؛ <code>TypeError/ValueError</code> ؛ مثال	<code>YieldCurve</code>

قرارداد API: هویت، قیمت، معاملات و API های قدیمی

```

normalize_instrument_text(value: 'Any') -> 'str'
resolve_instrument(selector=None, *, ins_code=None, asset_type='auto', snapshot=None,
require_active=True) -> 'InstrumentRef'
validate_ins_code(value: 'Any') -> 'str'
get_history(symbol='', start=None, end=None, limit=0, raw=False, auto_adjust=True,
output_type='standard', date_format='jalali', progress=True, save_to_file=False, dropna=True,
adjust_volume=False, return_type=None, ascending=True, save_path=None, include_today=False, *,
ins_code=None, asset_type='auto', **kwargs)
get_client_type(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, *, ins_code=None, asset_type='auto', **kwargs)
get_capital_increase(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_intraday(symbol='شتران', interval='1min', start=None, end=None, progress=True, **kwargs)
get_trades(symbol=None, *, ins_code=None, start=None, end=None, include_canceled=False,
max_requests=None, raw=False, progress=True)
get_live_trades(symbol=None, *, ins_code=None, include_canceled=False, max_requests=None, raw=False,
progress=True)
get_detail(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_info(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_stats(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_introduction(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_symbols(bourse=True, farabourse=True, payeh=True, haghe_taqadom=False, sandogh=False, bonds=False,
options=False, mortgage=False, commodity=False, energy=False, payeh_color=None, output='dataframe',
progress=True, **kwargs)
get_shareholders(symbol='', date=None, include_id=False, *, ins_code=None, asset_type='auto', **kwargs)
get_currency(name='', start=None, end=None, limit=0, output_type='standard', date_format='jalali',
progress=True, save_to_file=False, dropna=True, return_type=None, ascending=True, save_path=None,
**kwargs)
get_market_snapshot(*args, **kwargs)
get_market_client_type(*args, **kwargs)

```

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
<code>normalize_instrument_text</code>	به رشته تھی و <code>value: Any None</code> ؛ متن با یکسان‌سازی ی/ک، فاصله و به کلید مقایسه تبدیل می‌شود ZWNJ	<code>mutate</code> ؛ ورودی را canonicalize نمی‌کند	خطای ویژه ندارد؛ <code>normalize_instrument_text("کگل") == "کگل".xt</code>
<code>validate_instrument_code</code>	مثبت یا integer <code>value: str int</code> ؛ فاصله ابتدا/انتها) ASCII رشته ۰۰۱ رقم رقم فارسی/، float، bool، (می‌شود strip داخلی، صفر و طول space، sign، عربی بیش از ۲۰ رد می‌شوند	معتبر دقیقاً به ASCII: integer <code>str</code> ؛ رشته تبدیل می‌شود	<code>InvalidParameterError</code> ؛ resolver مثال بخش
<code>resolve_instrument</code>	<code>snapshot:</code> منبع <code>dict DataFrame None</code> ؛ حتی اگر تھی؛ <code>authoritative</code> ؛ <code>require_active: bool=True</code> ؛ ابزار غیرفعال را حذف می‌کند. اولویت آمده است resolver در فصل exact	<code>InstrumentRef</code> ؛ مسیر اثبات <code>provenance</code> ؛ <code>(explicit_instrument_code, snapshot/search/index registry</code> و مشابه) را ثبت می‌کند	<code>InvalidParameterError</code> ، <code>StockNotFoundError</code> ، <code>AmbiguousSymbolError</code> ، <code>DataParsingError</code> ، <code>ConnectionError</code> ؛ مثال: exact/ambiguity resolver.
<code>get_history</code>	<code>symbol/ins_code/asset_type</code> ؛ هویت؛ <code>start/end/limit/ascending</code> ؛ <code>raw</code> schema بازه و ترتیب؛ <code>OHLC</code> ؛ <code>auto_adjust</code> ؛ <code>adjust_volume</code> ؛ <code>output_type∈{standard,full}</code> ؛ <code>date_format∈{jalali,gregorian,both}</code> ؛ <code>return_type∈{simple,log,both}</code> ؛ <code>include_today</code> ؛ <code>progress/dropna</code> ؛ نمایش/ترکیب؛ <code>save_to_file/save_path</code> ؛ های alias فقط <code>kwargs</code> ؛ ذخیره مستند.	برای سهم و <code>auto_adjust=True</code> ، <code>output_type='standard'</code> ؛ دقیقاً <code>Open,High,Low,Close,Volume</code> ؛ <code>full</code> ؛ <code>me</code> ؛ <code>Final,Open,Value</code> ؛ <code>Ticker</code> ؛ <code>auto_adjust=False</code> ؛ <code>Adj Close</code> ؛ <code>standard</code> ؛ <code>index/industry schema</code> ؛ <code>MultiIndex</code> ؛ <code>include_today=True</code> ؛ <code>include_today_appended</code> ؛ <code>include_today_warning</code> ؛	ورودی‌های ناسازگار legacy معمولاً <code>ValueError / None</code> و <code>ConnectionError</code> ؛ مثال فصل قیمت.
<code>get_client_type</code>	<code>symbol/ins_code/asset_type</code> ؛ هویت؛ <code>start/end/limit/ascending</code> ؛ <code>raw</code> schema provider؛ <code>output_type∈{standard,full}</code> ؛ <code>date_format∈{jalali,gregorian,both}</code> ؛ <code>include_today</code> ؛ <code>progress/dropna</code> ؛ نمایش/چندنمادی؛ <code>save_to_file/save_path</code> ؛ های قدیمی alias؛ <code>kwargs</code> ؛ CSV؛	روزانه <code>DataFrame</code> ؛ <code>Buy_I/N_Count</code> ، <code>Buy_I/N_Volume</code> ، <code>Sell_I/N_Count</code> ، <code>Sell_I/N_Volume</code> ؛ قدرت/، <code>MultiIndex</code> ؛ <code>opt-in</code> ؛ <code>MultiIndex</code> ؛	خطای selector/provider یا legacy <code>None</code> ؛ مثال فصل حقیقی/حقوقی.
<code>get_capital_increase</code>	<code>stock=</code> ؛ قدیمی alias و selector	index یا <code>DataFrame None</code> ؛ <code>date</code> و <code>old_shares_amount,new_shares_amount</code> ؛	در provider/نماد گم‌شده/legacy <code>None</code> ؛ قرارداد قدیمی پیام و <code>att.get_capital_increase("فملی")</code> .

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
<code>get_intraday</code>	یکی از canonical <code>interval</code> <code>tick, 1min, 5min, 15min, 30min</code> های <code>alias</code> و <code>1h, 4h, 12h</code> مانند <code>_INTERVAL_MAP</code> <code>1m, 60min, 4hour, 240m, 12hour</code> بدون تاریخ snapshot <code>start</code> معاملات امروز، با تاریخ.	شامل <code>tick</code> ؛ <code>DataFrame None</code> شامل <code>candle</code> زمان/قیمت/حجم و <code>Open, High, Low, Close, Volume, TradeCount</code> با <code>DatetimeIndex</code> .	نامعتبر یا تاریخی که <code>interval</code> <code>validator</code> نامعتبر تشخیص <code>None</code> دهد پیام چاپ می‌کند و نماد نیز در مسیر/ <code>provider</code> می‌دهد؛ <code>ValueError</code> ؛ <code>None</code> ؛ <code>legacy</code> ؛ نیست؛ API خطای عمدی قرارداد این مثال <code>att.get_intraday("فملی", interval="5min")</code> .
<code>get_trades</code>	بدون تاریخ امروز تهران؛ <code>Thu/Fri</code> قبل از <code>budget</code> حذف؛ <code>include_canceled</code> ؛ <code>max_requests</code> فقط Trade endpoint؛ <code>raw</code> ستون‌های <code>provider</code> را اضافه می‌کند.	standard: <code>DataFrame[InsCode, Symbol, GregorianDate, JalaliDate, TradeNo, Time, Timestamp, Price, Volume, Value, Canceled, Source]</code> ؛ attrs شامل <code>request/provenance/failures</code> . <code>raw</code> نیز دارد <code>audit</code> ستون‌های	<code>InvalidParameterError</code> ، <code>resolver errors</code> ، <code>ConnectionError</code> ، مثال؛ <code>DataParsingError</code> ؛ فصل معاملات.
<code>get_live_trades</code>	هویت؛ <code>symbol/ins_code</code> رکورد ابطالی؛ <code>include_canceled</code> سقف درخواست؛ <code>max_requests</code> <code>raw</code> audit schema؛ <code>progress</code> نمایش. تاریخ به‌صورت خودکار امروز تهران است.	دقیق <code>attrs</code> و <code>schema</code> معاملات امروز با: <code>get_trades</code> زمان، قیمت، حجم، ارزش، وضعیت <code>provenance</code> ابطال و	<code>InvalidParameterError</code> ، <code>resolver errors</code> ، <code>ConnectionError</code> ، <code>DataParsingError</code> ؛ <code>att.get_live_trades(ins_code="...")</code> .
<code>get_detail</code>	قدیمی API HTML دقیق؛ <code>selector</code> .	با <code>index</code> <code>DataFrame None</code> به‌همراه، <code>value</code> و ستون <code>key</code> <code>row id</code> .	در رفتار <code>index/no-match/HTTP</code> مثال؛ <code>None</code> ؛ <code>legacy</code> <code>att.get_detail("فملی")</code> .
<code>get_info</code>	دقیق <code>selector</code> .	از <code>key/value</code> <code>DataFrame None</code> کامل <code>instrumentInfo</code> <code>flatten</code> .	یا <code>None</code> ؛ <code>legacy</code> ؛ مثال فصل <code>connection/parsing</code> ؛ اطلاعات نماد.
<code>get_stats</code>	دقیق <code>selector</code> .	<code>key/value</code> <code>DataFrame None</code> عددی <code>value</code> آمار با کلید فارسی و	یا <code>None</code> ؛ <code>legacy</code> <code>connection/parsing</code> ؛ <code>att.get_stats("فملی")</code> .
<code>get_introduction</code>	هیچ پارامتر BC فقط برای <code>signature</code> نمی‌شود I/O باعث	هرگز خروجی موفق ندارد.	همیشه <code>UnsupportedDataSourceError</code> پیش از I/O؛ جایگزین <code>market-data</code> ؛ <code>get_info/get_detail</code> .
<code>get_symbols</code>	<code>market/asset</code> ، های <code>boolean</code> ، <code>payeh_color</code> ؛ <code>output</code> ؛ <code>str list None</code> ؛ های <code>alias</code> ؛ <code>str='dataframe'</code> ؛ <code>kwargs</code> انگلیسی در	فهرست ابزارها یا <code>DataFrame</code> قدیمی انتخابی؛ ستون‌های <code>format</code> ؛ خروجی تهی <code>type</code> هویت/نام/بازار و پایدار دارد <code>schema</code> .	فیلتر <code>output</code> نامعتبر ؛ <code>ValueError</code> یا <code>None</code> ؛ <code>legacy</code> ؛ مثال فصل فهرست ابزارها.

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
<code>get_shareholders</code>	<code>include_id: bool=False</code> ; آخرین و تاریخ مشخص آن روز snapshot	<code>DataFrame None[share_holder_name,number_of_shares,percentage_of_shares,change_state,change_amount,date]</code> و با <code>opt-in share_holder_id</code> .	پیام و legacy در <code>provider/selector</code> مثال فصل اطلاعات نماد ; <code>None</code>
<code>get_currency</code>	نام ارز/سکه ; <code>name: str list</code> <code>start/end/limit/ascending</code> schema: <code>output_type</code> بازه ; <code>date_format∈{jalali,gregorian,both}</code> بازه ; <code>return_type</code> نمایش / <code>progress/dropna</code> چندارزی ; <code>save_to_file/save_path</code> های قدیمی. منبع <code>kwargs</code> alias CSV: است TGUJ	تک ارز <code>DataFrame[Open,High,Low,Close]</code> چند ارز ستون <code>MultiIndex</code> ; index تاریخ.	نام نامعتبر/ <code>provider</code> ممکن است مثال <code>ValueError / None</code> ; فصل ارز.
<code>get_market_snapshot</code>	به تابع BC برای <code>*args/**kwargs</code> <code>market_watch</code> صفرآرگومان می‌شود؛ در عمل آرگومان غیرتهی forward <code>TypeError</code> می‌دهد.	با کلیدهای دقیق <code>dict</code> <code>stocks,order_book,market_time,index_value,migration,trade_date,market_state,exchange_time,fetched_at,snapshot_age_seconds,is_today_trade_date,is_history_eligible,is_realtime_fresh,is_previous_trade_date,is_stale,is_partial</code> .	<code>ConnectionError</code> , مثال <code>DataParsingError</code> ; live.
<code>get_market_client_type</code>	<code>*args/**kwargs</code> wrapper <code>market_client_type</code> .	<code>DataFrame[InsCode,Buy_I_Count,...,Net_I_Volume,Net_N_Volume]</code> .	<code>ConnectionError</code> , مثال <code>DataParsingError</code> ; live.

قرارداد live: API، سفارش، watcher و تحلیل بازار

```

get_order_book(symbol=None, *, selector_strict=False)
get_live_market(symbol=None, *, strict=False)
get_live_symbol(symbol=None, *, ins_code=None, fallback='none')
get_order_book_history(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, complete_only=False, *, max_requests=None, _request_budget_state=None, **kwargs)
get_orderbook_history(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, complete_only=False, *, max_requests=None, _request_budget_state=None, **kwargs)
get_queue(symbol=None, side='both', strict=True, *, selector_strict=False)
get_queue_history(symbol='', start=None, end=None, limit=0, date_format='jalali', progress=True,
save_to_file=False, dropna=True, ascending=True, save_path=None, include_today=False,
complete_only=False, side='both', strict=True, *, max_requests=None, **kwargs)
MarketWatcher(symbol=None, interval=1.0, max_updates=None, include_initial=True, emit_heartbeats=True,
notifications=('messages', 'state'), notification_top=50, stop_event=None, error_policy='retry',
max_backoff=30.0, jitter=0.1, callback_error_policy='raise', max_consecutive_retries=5,
max_retries=None, max_seen_notifications=10000, copy_snapshot=True, request_timeout=None, *,
record_to=None, record_heartbeats=False, checkpoint_interval=100, record_max_records=10000,
record_retention_seconds=None, storage_error_policy='raise', _request=safe_get, _clock=None,
_wait=None, _random=None)
watch_market(*args, **kwargs)
get_market_messages(flow=0, top=20, since_id=None, *, archive_to=None, _recorded_at=None,
_request=safe_get)
get_instrument_state_changes(top=20, since_id=None, *, archive_to=None, _recorded_at=None,
_request=safe_get)
get_market_overview(flow=0, *, archive_to=None, _recorded_at=None, _request=safe_get)
get_market_breadth(symbol=None, flow=None, sector=None, traded_only=False, include_base_market=True,
instrument_types=None, *, _snapshot=None)
get_sector_flow(symbol=None, flow=None, sector=None, traded_only=False, include_base_market=True,
instrument_types=None, *, _snapshot=None, _client_type=None)

```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
get_order_book	scalar/list/ None ; فقط selector_strict انتخاب را سخت می‌کند resolution.	با long DataFrame InsCode, Symbol, Name, Level, BidOrderCount, BidVolume, BidPrice, AskPrice, AskVolume, AskOrderCount و metadata trade_date, market_state, exchange_time, fetched_at, is_* response پنج سطح از همان .	InvalidParameterError , StockNotFoundError در strict. connection/parsing مثال: فصل سفارش.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_live_market</code>	های <code>selector</code> <code>strict=False</code> گم‌شده را در <code>attrs['missing_selectors']</code> آن‌ها را خطا می‌کند <code>strict</code> ثبت می‌کند؛ <code>]</code>	با <code>DataFrame</code> <code>STOCK_COLUMNS</code> ، <code>client</code> <code>columns</code> ، <code>wide</code> سطح‌های <code>BidPrice1..5/AskPrice1..5</code> از جمله <code>metrics</code> و <code>EstimatedNetIndividualFlow</code> ؛ <code>freshness</code> ستون <code>row-wise</code> ؛ <code>attrs</code> دقیق است: <code>field_validity, source_schema_presence, migration, missing_selectors</code> . <code>Last</code> پایانی است <code>Close</code> آخرین معامله و	<code>InvalidParameterError</code> ، <code>StockNotFoundError</code> در <code>strict</code> ، <code>ConnectionError/DataParsingError</code> ؛ <code>quickstart</code> و مثال <code>attrs</code> بالا.
<code>get_live_symbol</code>	<code>fallback: 'none' 'point'</code> ؛ <code>point</code> فقط در غیاب <code>MarketWatch</code> . <code>fallback='none'</code> برای <code>no-match</code> <code>frame</code> نمی‌دهد.	exception؛ دقیقاً یک‌ردیفی یا <code>frame</code> <code>provenance</code> <code>SnapshotSource='market_watch' 'closing_price_info'</code> و ستون‌های <code>_fallback'</code> <code>PresentInMarketWatch, MarketStateTitle, has_trade_today, PriceActionable, fallbackReason, IdentityVerified</code> .	<code>no-match</code> با <code>fallback none:</code> <code>StockNotFoundError</code> ؛ همچنین <code>InvalidParameterError</code> ، <code>AmbiguousSymbolError</code> ، <code>DataParsingError</code> ؛ فصل <code>live</code> .
<code>get_order_book_history</code>	<code>raw</code> ؛ <code>output_type='standard'</code> <code>long</code> و <code>'wide'</code> ؛ <code>complete_only</code> ؛ <code>budget</code> این <code>calls</code> همه <code>workflow</code> .	<code>long/wide/raw DataFrame</code> ؛ <code>identity/time/5</code> سطح و <code>is_partial, is_complete, market_partial_status, is_reconstructed, is_stale, record_type, source</code> ؛ <code>attrs</code> <code>schema, failed_requests, request_count, max_requests</code> .	<code>InvalidParameterError</code> ، <code>DataParsingError</code> ، <code>resolver/connection: failure</code> ؛ مثال فصل <code>attrs</code> + <code>warning</code> جزئی تاریخچه سفارش.
<code>get_orderbook_history</code>	کاملاً یکسان با <code>signature</code> هویتی و <code>alias</code> <code>get_order_book_history</code> .	دقیقاً همان <code>.object/return/schema/attrs</code>	همان خطاها و همان مثال؛ <code>att.get_orderbook_history</code> <code>is</code> <code>att.get_order_book_history</code> <code>.ry</code>
<code>get_queue</code>	فقط <code>side</code> ؛ <code>strict=True</code> ؛ <code>queue</code> می‌دارد <code>queue</code> های قطعی را نگه می‌دارد.	<code>DataFrame[InsCode, Symbol, Name, ..., Side, QueuePrice, QueueVolume, QueueOrders, QueueValue, PriceLimit, is_queue, book_state, ...]</code> .	نامعتبر <code>side</code> ؛ <code>InvalidParameterError</code> ؛ مثال <code>selector/provider errors</code> صف.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_queue_history</code>	<code>symbol/start/end/limit/asc</code> تاریخ؛ <code>date_format</code> بازه؛ <code>include_today</code> live؛ <code>complete_only</code> snapshot؛ کامل؛ <code>side∈{buy,sell,both}</code> <code>strict</code> فقط صف قطعی؛ <code>max_requests</code> بودجه؛ <code>progress/dropna</code> نمایش/تهی؛ <code>save_to_file/save_path</code> از CSV؛ <code>kwargs</code> alias. threshold و است as-of snapshot همان تاریخ و	<code>QUEUE_COLUMNS</code> <code>is_queue</code> nullable. crossed book false؛ attrs budget/failures.	<code>InvalidParameterError</code> , resolver/connection/parsing؛ مثال فصل صف
<code>MarketWatcher</code>	فقط <code>notifications</code> ها؛ policy enum؛ glossary دقیقاً در <code>interval,max_backoff>=0</code> , نامنفی <code>retry limit</code> , <code>0<=jitter<=1</code> , <code>request_timeout/record_reten</code> <code>tion_seconds>0</code> ; دارای <code>stop_event</code> opt-in. <code>is_set/wait</code> ; <code>record_to</code>	<code>MarketEvent</code> ; سنکرون کند؛ <code>initial/delta/heartbeat/</code> <code>resync</code> ; خود constructor I/O نمی‌کند.	نامعتبر policy/range <code>InvalidParameterError</code> iteration: runtime <code>ConnectionError/DataPar</code> <code>singError</code> : مثال policy طبق watcher.
<code>watch_market</code>	تمام <code>*args/**kwargs</code> بدون تغییر به <code>MarketWatcher</code> می‌روند.	نه ، <code>MarketWatcher</code> DataFrame.	همان constructor/runtime؛ مثال <code>for event in</code> <code>att.watch_market(max_upd</code> <code>.ates=3): ...</code>
<code>get_market_messages</code>	local؛ فیلتر <code>since_id</code> opt-in. نوشتن اتمیک <code>archive_to</code>	<code>DataFrame[message_id,date,time,timestamp,title,description,flow]</code> ; attrs provenance/archive.	<code>InvalidParameterError</code> , <code>ConnectionError</code> , <code>DataParsingError</code> , storage error؛ مثال فصل پیام
<code>get_instrument_state_changes</code>	<code>since_id</code> bounded؛ تعداد <code>top</code> فقط رکوردهای بعد از شناسه؛ opt-in؛ SQLite مسیر <code>archive_to</code> فقط <code>_recorded_at/_request</code> seam تست.	<code>DataFrame[event_id,date,time,timestamp,InsCode,Symbol,Name,state_code,state_real_time,under_supervision,state_title]</code> .	<code>InvalidParameterError</code> , <code>ConnectionError</code> , <code>DataParsingError</code> یا خطای storage؛ مثال فصل وضعیت
<code>get_market_overview</code>	تهی. صفر ردیف است <code>payload</code> ; <code>flow</code>	با <code>DataFrame</code> overview payload/fast-view؛ فیلدهای <code>flow</code> ; attrs source/time/archive.	parameter/connection/parsing/storage؛ مثال overview.
<code>get_market_breadth</code>	فیلترهای universe؛ denominator ابرار انتخاب شده.	یک ردیف <code>DataFrame</code> با counts/percentages. A/D، و volume/value. limit counts .freshness. attrs analytics/source	<code>InvalidParameterError</code> , <code>DataParsingError</code> ; مثال breadth.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_sector_flow</code>	فیلتر: <code>symbol/flow/sector</code> <code>traded_only</code> universe معامله‌شده؛ بازار پایه: <code>include_base_market</code> کد نوع ابزار: <code>instrument_types</code> فقط <code>_snapshot/_client_type</code> فقط برای ردیف client feed. تست شده به کار می‌رود <code>reconcile</code>	یک ردیف در هر <code>SectorCode</code> با <code>breadth +</code> <code>client_coverage,net_individual_volume,estimated_net_individual_value,value</code> <code>_available,value_method</code> و <code>.freshness</code>	<code>parameter/connection/parsing:</code> مثال <code>sector</code> .

قرارداد API: تاریخچه محلی و fundamentals

```

get_market_fundamentals(symbols=None, *, pe_min=None, pe_max=None, positive_pe=True, instrument_types=(300, 303, 309), strict=False, allow_stale=False, archive_to=None, max_requests=1, progress=True)
get_market_fundamentals_history(path, start=None, end=None, symbols=None, *, pe_min=None, pe_max=None, positive_pe=True, instrument_types=(300, 303, 309), strict=False, allow_stale=True, limit=1000, offset=0)
get_price_adjustments(symbol=None, *, ins_code=None, start=None, end=None, progress=True)
get_latest_price_adjustment(symbol=None, *, ins_code=None, start=None, end=None, progress=True)
check_market_history(path)
save_market_snapshot(path, snapshot=None, *, as_of=None, _live_fetch=None)
load_market_snapshots(path, start=None, end=None, symbol=None, *, limit=1000, offset=0)
get_live_market_history(path, start=None, end=None, symbol=None, *, limit=1000, offset=0)
get_market_overview_history(path, start=None, end=None, flow=0, *, source='tsetmc', limit=1000, offset=0)
get_market_snapshot_summary_history(path, start=None, end=None, flow=None, *, limit=1000, offset=0)
get_market_breadth_history(path, start=None, end=None, symbol=None, flow=None, sector=None, traded_only=False, include_base_market=True, instrument_types=None, *, limit=1000, offset=0)
get_sector_flow_history(path, start=None, end=None, symbol=None, flow=None, sector=None, traded_only=False, include_base_market=True, instrument_types=None, *, limit=1000, offset=0)
record_market_event(path, event, *, session_id=None, include_snapshot=True, max_records=10000, retention_seconds=None)
get_market_event_history(path, start=None, end=None, kind=None, *, session_id=None, limit=1000, offset=0)
archive_market_records(path, kind, frame, *, source='tsetmc', recorded_at=None)
get_market_messages_history(path, start=None, end=None, flow=0, since_id=None, *, source='tsetmc', limit=1000, offset=0)
get_instrument_state_changes_history(path, start=None, end=None, symbol=None, since_id=None, *, inscode=None, source='tsetmc', limit=1000, offset=0)

```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_market_fundamentals</code>	<code>pe_min/pe_max: float None</code> <code>positive_pe=True</code> شامل مرز؛ <code>archive_to</code> ؛ مثبت؛ P/E فقط <code>max_requests=1</code> bulk.	دقیق attrs ثابت بالا: <code>schema</code> <code>request_count,max_requests,missing_selectors,strict,stale_rejected,source,price_source,eps_source,no_lookahead,no_backfill,archive_path</code> .	<code>bounds/filter/no-match</code> در <code>strict:</code> <code>InvalidParameterError</code> <code>/resolver error: stale</code> با <code>allow_stale=False</code> frame <code>exception:</code> نه attr تهی و مثال <code>provider typed errors</code> . <code>fundamentals</code> .

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_market_fundamentals_history</code>	<code>path</code> SQLite؛ <code>start/end/limit/offset</code> فیلتر؛ <code>symbols</code> صفحه و بازه؛ <code>pe_min/pe_max/positive_pe</code> <code>instrument_types</code>؛ <code>P/E</code>؛ <code>universe</code>؛ <code>strict</code> <code>no-match</code>؛ <code>allow_stale</code> snapshot نگه‌داشتن قدیمی.	<code>attrs</code> زنده با <code>schema</code> <code>missing_selectors,strict</code> , <code>source,price_source,eps_source,current_eps_used,no_lookahead,no_backfill,coverage_start,coverage_end</code> ; <code>current_eps_used=False</code> .	<code>file/schema/filter</code> یا <code>InvalidParameterError</code> ; مثال <code>DataParsingError</code> ; <code>backtest</code>.
<code>get_price_adjustments</code>	شامل <code>date bounds</code> دقیق و <code>selector</code>.	<code>DataFrame[InsCode,Symbol, GregorianDate, JalaliDate, AdjustedClosingPrice, UnadjustedClosingPrice, AdjustmentAmount, CorporateTypeCode, CorporateActionType, IsConfirmedDPS, IdentityVerified, Source, FetchedAt]</code> ; <code>IsConfirmedDPS=False</code> , <code>attrs dps_available=False</code> .	<code>resolver/parameter/connection/parsing</code>؛ مثال فصل تعدیل
<code>get_latest_price_adjustment</code>	هویت؛ <code>symbol/ins_code</code>؛ فیلتر تاریخ؛ <code>start/end</code>؛ نمایش. پس از اعمال فیلتر <code>progress</code> فقط جدیدترین رخداد انتخاب می‌شود.	<code>schema</code> <code>get_price_adjustments</code> , <code>attrs</code> و <code>typed</code> صفر یا یک ردیف <code>provenance</code>.	<code>InvalidParameterError</code> , <code>resolver errors</code>, <code>ConnectionError</code> , <code>DataParsingError</code> ; مثال فصل تعدیل
<code>check_market_history</code>	موجود باشد SQLite باید <code>path</code>.	دقیقاً <code>dict[str, bool int]</code> با <code>ok=True, SchemaVersion, ApplicationID</code> <code>DataFrame</code> نیست.	فایل گم‌شده ؛ <code>InvalidParameterError</code> ; غیر SQLite/نسخه ناسازگار <code>DataParsingError</code> ; مثال <code>att.check_market_history(db)</code>
<code>save_market_snapshot</code>	<code>snapshot</code>؛ اگر <code>DataFrame dict None</code> ; <code>as_of</code> فقط یک <code>None</code> مشاهده <code>override</code>.	<code>str</code> , <code>SHA-256 snapshot_id</code> ; همزمان و <code>live/client/order</code> با <code>DB</code> داخل <code>attrs</code> اتمیک؛ <code>transaction</code>؛ <code>reader</code> می‌شود بازیابی.	<code>path/type/schema/storage</code> <code>InvalidParameterError/DataParsingError</code> ; <code>network</code> ; مثال <code>snapshot=None</code> فقط هنگام SQLite.
<code>load_market_snapshots</code>	فیلتر <code>inclusive</code> زمان و <code>symbol</code>؛ <code>pagination SQL</code>.	ردیف‌های <code>DataFrame</code> با <code>STOCK_COLUMNS</code> <code>observation</code> و <code>meta</code> <code>SnapshotID, AsOf, Source, SchemaVersion, NoBackfill, CoverageStart, SnapshotAtomic, PersistenceAtomic, SourceAtomic, PriceSourceAsOf, ClientSourceAsOf, NoLookahead</code> .	؛ مثال <code>typed</code> <code>file/schema/filter</code> SQLite.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_live_market_history</code>	<code>path</code> SQLite؛ <code>start/end/symbol/limit/off</code> <code>set/ascending/include_stal</code> <code>e</code> معنایی <code>alias</code> برای فیلتر و صفحه‌بندی <code>reader rich live</code> alias <code>identity</code> است، نه	سازگار با <code>frame/meta</code> <code>load_market_snapshots</code>	<code>InvalidParameterError</code> و <code>file/schema</code> خطاهای <code>att.get_live_market_his</code> <code>tory(db,</code> <code>symbol="فملی")</code>
<code>get_market_overview_history</code>	<code>flow/source</code> ، مستقل <code>archive</code>	+ قبلی <code>payload exact provider</code> <code>meta archive</code> <code>ArchiveIdentity,Version,</code> <code>FirstObservedAt,ObservedA</code> <code>t,ProviderTimestamp,AsOf,</code> <code>Source,SchemaVersion,NoBa</code> <code>ckfill,...</code>	<code>kind/source/schema</code> نامعتبر <code>typed</code> ؛ <code>archive_to</code> ؛ مثال
<code>get_market_snapshot_summary_history</code>	<code>snapshot atomic</code> مشتق از <code>summary</code> ؛ <code>flow=None</code> همه	<code>DataFrame[flow,instrumen</code> <code>t_count,trade_count,total</code> <code>_volume,total_value,marke</code> <code>t_cap,trade_date,exchange</code> <code>_time,fetched_at,is_realt</code> <code>ime_fresh]</code> + <code>meta</code>	<code>file/filter/schema errors</code> ؛ مثال <code>SQLite</code>
<code>get_market_breadth_history</code>	همان فیلترهای <code>live</code> روی هر <code>snapshot</code> <code>.persisted</code>	<code>BREADTH_COLUMNS</code> + <code>meta</code> ؛ <code>snapshot-by-snapshot</code> ، بدون <code>look-ahead</code>	<code>file/filter/schema errors</code> ؛ مثال <code>breadth history</code>
<code>get_sector_flow_history</code>	همان فیلترهای <code>live</code> ، <code>prices</code> و <code>client</code> ، همان <code>observation</code>	<code>SECTOR_FLOW_COLUMNS</code> + <code>meta/no-lookahead</code>	<code>file/filter/schema errors</code> ؛ مثال <code>sector history</code>
<code>record_market_event</code>	<code>event: MarketEvent</code> ؛ <code>include_snapshot</code> ؛ <code>retention/session</code>	<code>str</code> ، <code>SHA-256</code> <code>event_id</code> ؛ <code>event</code> و <code>notification/snapshot</code> <code>transaction</code>	<code>type/kind/storage</code> <code>InvalidParameterError/D</code> <code>ataParsingError</code> ؛ مثال <code>record_to</code> <code>watcher</code>
<code>get_market_event_history</code>	<code>kind=None initial delta he</code> <code>artbeat resync</code> ؛ <code>session/pagination</code>	<code>DataFrame[event_id,Sessi</code> <code>onID,kind,SnapshotMode,se</code> <code>quence,fetched_at,trade_d</code> <code>ate,changed_inscodes,chan</code> <code>ged_order_levels,market_s</code> <code>tate_changed,notification</code> <code>_tokens,cursor_before,cu</code> <code>rsor_after,retry_count,ret</code> <code>ry_error,notification_err</code> <code>ors,snapshot,messages,sta</code> <code>te_changes]</code> + <code>meta</code>	<code>kind</code> ناشناخته <code>InvalidParameterError</code> <code>schema</code> حتی برای فایل گم‌شده؛ <code>replay</code> ؛ مثال <code>errors</code>
<code>archive_market_records</code>	یکی از <code>kind</code> <code>messages,state_changes,ove</code> <code>rview frame: DataFrame</code> ؛ <code>recorded_at/source</code> هویت <code>archive</code>	های تاریخچه <code>version</code> تعداد <code>int</code> <code>duplicate</code> <code>true</code> نوشته‌شده؛ <code>transaction atomic</code>	<code>kind/type/storage errors</code> ؛ مثال <code>helper</code> های <code>archive_to</code>

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_market_messages_history</code>	و <code>flow/since_id/source</code> و <code>time/page filters</code> .	schema + پیام archive meta: dedup provider ID.	؛ مثال: <code>file/schema/filter errors</code> archive.
<code>get_instrument_state_changes_history</code>	قدیمی spelling و/یا <code>symbol</code> <code>inscode</code> ; <code>since_id/source</code> .	schema state + archive meta.	؛ مثال: <code>selector</code> متناقض/فایل <code>schema errors</code> archive.

قرارداد API: درآمد ثابت

```

parse_treasury_maturity(symbol)
day_count_fraction(start, end, convention='ACT/365F')
treasury_yield(price, maturity_date, settlement_date=None, face_value=1000000.0, day_count='ACT/365F')
bond_price(annual_yield, cashflows=None, settlement_date=None, day_count='ACT/365F',
compounding='nominal', frequency=1, price_type='dirty', accrued_interest=None, maturity_date=None,
face_value=None, coupon_rate=None, issue_date=None)
yield_to_maturity(price, cashflows=None, settlement_date=None, day_count='ACT/365F',
compounding='nominal', frequency=1, price_type='dirty', accrued_interest=None, maturity_date=None,
face_value=None, coupon_rate=None, issue_date=None, tolerance=1e-12, max_iterations=300)
bond_analytics(price, cashflows=None, settlement_date=None, day_count='ACT/365F',
compounding='nominal', frequency=1, price_type='dirty', accrued_interest=None, maturity_date=None,
face_value=None, coupon_rate=None, issue_date=None, annual_yield=None, bump_size=0.0001)
build_yield_curve(nodes, settlement_date, interpolation='log_discount', extrapolate=False,
duplicate_policy='error', day_count='ACT/365F', rate_compounding='effective', rate_frequency=1,
enforce_monotonic_discount=True)
get_ifb_yield_table(category='treasury')
get_treasury_yields(symbol=None, settlement_date=None, face_value=1000000.0, include_stale=False,
min_volume=0, price_source='auto', day_count='ACT/365F', strict=False, face_value_source=None,
source='tsetmc', maturity_date=None, maturity_map=None, allow_no_trade=False)
get_treasury_yield_history(symbol=None, start=None, end=None, limit=0, settlement_date=None,
face_value=1000000.0, include_today=False, date_format='jalali', price_source='auto',
day_count='ACT/365F', ascending=True, progress=True, strict=False, face_value_source=None,
max_requests=250, maturity_date=None, maturity_map=None, use_ifb_reference=False)
get_treasury_yields_history(symbol=None, start=None, end=None, limit=0, settlement_date=None,
face_value=1000000.0, include_today=False, date_format='jalali', price_source='auto',
day_count='ACT/365F', ascending=True, progress=True, strict=False, face_value_source=None,
max_requests=250, maturity_date=None, maturity_map=None, use_ifb_reference=False)
get_yield_curve(symbol=None, settlement_date=None, face_value=1000000.0, include_stale=False,
min_volume=0, min_nodes=3, price_source='auto', day_count='ACT/365F', interpolation='log_discount',
extrapolate=False, duplicate_policy='volume_weighted', enforce_monotonic_discount=True,
source='tsetmc')
get_yield_curve_history(symbol=None, start=None, end=None, limit=0, face_value=1000000.0,
include_today=False, date_format='jalali', price_source='auto', day_count='ACT/365F', min_nodes=3,
interpolation='log_discount', duplicate_policy='volume_weighted', enforce_monotonic_discount=True,
ascending=True, progress=True, max_requests=250, maturity_map=None)

```

خطا و مثال	خروجی/schema/attrs	ورودی‌ها و گزینه‌ها	تابع
non-match/تاریخ نامعتبر/ None ، deterministic ؛ مثال exception نه بالاتر.	موفق: dict None ؛ maturity_jalali: str ، maturity_gregorian: datetime.date ، maturity_source='user_confirmed_symbol_jalali_yymmdd' .	symbol: Any ؛ فقط full-match پس از تبدیل رقم IYYMMDD خزا ZWNJ؛ فارسی/عربی و حذف و 00..79→1400..1479 80..99→1380..1399 .	parse_treasury_maturity
نامعتبر date order/convention ValueError ؛ day_count_fraction(date(2025,1,1), date(2026,1,1)) == 1.0 .	float year fraction.	start/end: date-like ؛ convention .	day_count_fraction
face/date/day-count/قیمت نامعتبر/ ValueError ؛ مثال deterministic .	شامل dict DiscountFactor, EffectiveAnnualYield, ContinuousYield, SimpleAnnualYield, BankDiscountYield, MacaulayDuration, ModifiedDuration, Convexity, DV01, DaysToMaturity/Tenor .	zero-coupon price/face/maturity/settlement.	treasury_yield
cashflow/rate/frequency/date مثال: ValueError ؛ نامعتبر bond_price(0.2, [(date(...), amount)], ...)	float clean یا dirty price.	یا cashflows صریح، یا پارامترهای ساخت ؛ schedule annual_yield ؛ price_type .	bond_price
price bounds/عدم bracket خارج price ؛ ValueError ؛ یا عدم همگرایی فصل درآمد ثابت round-trip مثال	float annual yield با انتخابی compounding.	قیمت مشاهده شده؛ price صریح یا cashflows settlement_date/maturity_date/face_value/coupon_rate/frequency/issue_date برای schedule؛ accrued_interest/price_type clean/dirty؛ ؛ compounding tolerance/max_iterations solver.	yield_to_maturity
cashflow/rate/date/price نامعتبر ValueError ؛ ناموفق solver یا deterministic مثال	dict با price/yield. MacaulayDuration, ModifiedDuration, Convexity, DV01 و metadata schedule.	ورودی schedule مانند ؛ yield_to_maturity annual_yield=None یعنی حل ؛ price از YTM day_count/compounding/price_type convention bump_size=0.0001 شوک DV01.	bond_analytics

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>build_yield_curve</code>	node های rate/discount، compounding، duplicate و monotonic guard.	<code>YieldCurve</code> ; و <code>node_metadata</code> با کلیدهای دقیق <code>diagnostics</code> <code>input_node_count, node_count, duplicate_count, duplicate_policy, monotonic_discount_enforced</code> .	node کم/duplicate/discount غیرمثبت/non-monotonic <code>ValueError</code> ; مثال <code>curve</code> .
<code>get_ifb_yield_table</code>	<code>category: str='treasury'</code> IFB. دسته جدول صفحه	<code>DataFrame[Symbol, Price, LastTradeJalali, LastTradedate, PublishJalali, PublishDate, MaturityJalali, Maturity, Volume, ReferenceYTM, ReferenceSimpleYield, ReferenceSource]</code> ; attrs <code>provenance URL/time</code> .	category/HTML/schema/connection typed؛ مثال comparison IFB.
<code>get_treasury_yields</code>	live universe: <code>min_volume</code> ; <code>face_value_source</code> ; maturity override/map: <code>allow_no_trade</code> ; source.	هویت: <code>TREASURY_COLUMNS</code> ; price provenance، سررسید/تسویه yield، چهار duration/convexity/DV01، stale/status: attrs source/universe/fetch time.	<code>InvalidParameterError</code> , <code>StockNotFoundError/AmbiguousSymbolError</code> , <code>ConnectionError/DataParsingError</code> ; مثال <code>live</code> اخرا.
<code>get_treasury_yield_history</code>	history OHLC: <code>max_requests</code> فقط <code>use_ifb_reference</code> reference: settlement per row مگر override.	<code>TREASURY_HISTORY_COLUMNS</code> با <code>S</code> <code>TradeDate, JalaliDate, Open, High, Low, Close, Final, Volume...</code> و analytics: attrs failures/request/provenance.	parameter/resolver/provider/parsing: partial در attrs/warning history؛ مثال history.
<code>get_treasury_yields_history</code>	کامل یکسان signature با alias identity	object/schema/attrs همان <code>get_treasury_yield_history</code>	همان؛ <code>att.get_treasury_yields_history</code> is <code>att.get_treasury_yield_history</code>
<code>get_yield_curve</code>	<code>min_nodes</code> ; duplicate حداقل :default volume-weighted .extrapolation opt-in	از <code>YieldCurve</code> calibrated و snapshot live: diagnostics metadata nodeها را ثبت <code>provenance/no-lookahead</code> می‌کند.	node ناکافی/invalid curve <code>ValueError</code> و provider typed: مثال <code>curve live</code> .
<code>get_yield_curve_history</code>	trade date: جدا برای هر curve بدون budget، hard عمومی extrapolate.	panel <code>DataFrame</code> با <code>CURVE_HISTORY_COLUMNS</code> , <code>CurveID, CurveStatus, CurveError, CurveNodeCount</code> و flags no-lookahead: attrs failures/request.	parameter/budget/provider/curve errors؛ روز ناموفق در status/attrs: مثال <code>curve history</code> .

قرارداد API: شاخص‌های صنعت

```
list_industry_indices(progress=True, include_member_count=False, refresh=False, max_workers=6)
get_industry_members(industry, include_live=True, include_client_type=False, include_orderbook=False,
progress=True, refresh=False)
get_industry_snapshot(industries=None, include_client_type=True, include_orderbook=False,
include_empty=False, progress=True, refresh=False, max_workers=6)
get_industry_history(industry, start=None, end=None, limit=0, ascending=True, progress=True)
get_industry_members_history(industry, days=30, ascending=True, progress=True, refresh=False)
get_industry_intraday(industry, interval='1min', progress=True)
rank_industries(metric='IndexChangePct', top=None, ascending=False, include_client_type=True,
include_orderbook=False, progress=True, refresh=False, max_workers=6)
```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>list_industry_indices</code>	پیام: <code>progress: bool=True</code> عضویت ۴۵ صنعت را <code>include_member_count: bool=False</code> واکشی می‌کند: <code>refresh: bool=False</code> را دور می‌زند: <code>cache</code> در بازهٔ <code>max_workers: int=6</code> 1..16 .	<code>DataFrame[IndustryName, IndustryNameEn, IndustryGroupCode, IndexInsCode, IndexValue, IndexPreviousValue, DayHigh, DayLow, IndexChange, IndexChangePct, MemberCount, HasMembers, ExchangeTime]</code> attrs source/cache/count.	نامعتبر <code>bool/worker</code> : <code>InvalidParameterError</code> provider : <code>ConnectionError/DataParseError</code> مثال فصل صنعت
<code>get_industry_members</code>	نام/alias/ <code>industry: str int</code> : <code>include_live=True</code> MarketWatch: : <code>include_client_type=False</code> client metrics: پنج <code>include_orderbook=False</code> دو . <code>refresh=False</code> سطح/صف: نیاز دارند به live به enrichment گزینه	<code>INDUSTRY_MEMBER_COLUMNS</code> ثابت یا هویت، قیمت، معامله market cap تخمینی، client، order book و freshness: attrs exact membership/current/cache/coverage request.	صنعت گم‌شده : <code>StockNotFoundError</code> ترکیب/نوع نامعتبر : <code>InvalidParameterError</code> مثال typed provider: با <code>get_industry_members("نک")</code>
<code>get_industry_snapshot</code>	: <code>industries: None str int iterable=None</code> : <code>include_client_type=True</code> : <code>include_orderbook=False</code> : <code>include_empty=False</code> : <code>refresh=False</code> : <code>max_workers=6</code>	یک ردیف در هر شاخص با <code>INDUSTRY_SNAPSHOT_COLUMNS</code> S: وضعیت شاخص، breadth، equal-weight/median/dispersion معامله، flow/coverage، freshness و queue/coverage attrs overlap/current membership/cache	selector/bool/worker یا <code>InvalidParameterError</code> : <code>StockNotFoundError</code> snapshot/schema/provider typed: مثال فصل صنعت
<code>get_industry_history</code>	: <code>industry</code> اجباری start/end: <code>limit: str None</code> شمسی/میلادی: : <code>ascending: int=0</code> پس از فیلتر: : <code>bool=True</code> ; <code>progress</code> .	<code>DataFrame[IndustryName, IndustryIndexCode, TradeDate, JalaliDate, High, Low, Close, Change, ChangePct]</code> ; attrs <code>volume_available=False, completed_sessions_only=True</code> .	تاریخ/limit/order نامعتبر : <code>InvalidParameterError</code> صنعت/typed schema/provider: مثال تاریخچه.

خطا و مثال	خروجی/schema/attrs	ورودی‌ها و گزینه‌ها	تابع
days/bool/selector typed: provider/schema typed مثال: فصل صنعت	long-form INDUSTRY_MEMBER_HISTORY_ COLUMNS attrs برای اعضای فعلی; point_in_time_membership =False,survivorship_bias_ possible=True .	: industry days: int=30 تاریخ آخر؛ 1..30 دقیقاً ascending=True ; refresh=False .	get_industry_me mbers_history
interval/selector InvalidParameterError/S :tockNotFoundError provider/schema typed مثال: get_industry_intraday("minخودرو", 5) .	DataFrame[IndustryName,I ndustryIndexCode,Timestam p,JalaliDate,Interval,Ope n,High,Low,Close,Change,C hangePct] ; timezone، تهران attrs مصنوعی و بدون volume latest-day.	: industry interval: یکی از str='1min' raw,1min,5min,15min,30min, 60min,1h ; progress .	get_industry_in traday
metric/top/bool/worker نامعتبر : InvalidParameterError provider typed مثال: rank_industries(metric="breadth", top=5) .	;snapshot + Rank تمام ستون‌های attrs ردیف metric تهی حذف; ranking_metric,ranking_a provenance و scending .snapshot	مستند؛ canonical/alias metric top: int None ; ascending=False ; client/order switches: refresh ; max_workers .	rank_industrie s

قرارداد API: اختیار معامله و فهرست ابزارها

```

list_options(underlying=None, progress=True)
get_options_chain(underlying, fetch_oi=False, progress=True)
black_scholes_price(spot, strike, time_to_expiry, rate, volatility, option_type='call',
dividend_yield=0.0, exercise_style='european')
black_scholes_greeks(spot, strike, time_to_expiry, rate, volatility, option_type='call',
dividend_yield=0.0, exercise_style='european')
option_price_bounds(spot, strike, time_to_expiry, rate, option_type='call', dividend_yield=0.0,
exercise_style='european')
implied_volatility(option_price, spot, strike, time_to_expiry, rate, option_type='call',
dividend_yield=0.0, exercise_style='european', lower_volatility=0.0, upper_volatility=5.0,
tolerance=1e-08, max_iterations=200)
get_option_market(exchange=0, underlying=None, progress=True, max_requests=1)
analyze_option_chain(options=None, underlying=None, spot=None, risk_free_rate=None, yield_curve=None,
dividend_yield=0.0, valuation_date=None, exercise_style='european', parity_tolerance=None,
liquidity_weights=None, progress=True, *, allow_unverified_freshness=True)
option_put_call_ratios(options, group_by='market')
get_option_history(symbol, start=None, end=None, limit=0, include_today=False, snapshot_path=None,
progress=True, max_requests=3)
save_option_snapshot(path, options=None, exchange=0, progress=True, *, lock_timeout=10.0,
stale_lock_seconds=300.0)
load_option_snapshots(path)
list_etfs(progress=True)
list_bonds(progress=True)
list_funds(fund_type=None, progress=True, *, listed_only=False)
list_listed_funds(progress=True)
list_indices(progress=True)
get_index_companies(index_name, progress=True)

```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>list_options</code>	<code>underlying: str None</code> filter نام underlying.	قراردادها با هویت <code>DataFrame</code> ، <code>OptionType, Underlying*, Strike, BeginDate, EndDate, DaysToExpiry, ContractSize</code> و قیمت/حجم.	یا <code>provider parse/connection</code> frame تهی؛ مثال فصل اختیار
<code>get_options_chain</code>	اجباری؛ <code>underlying</code> fan-out OI از <code>fetch_oi=False</code> جلوگیری می‌کند.	<code>calls:</code> دقیقاً شامل <code>dict DataFrame, puts: DataFrame, underlying_name, underlying_price, expiry_dates, market_time</code> ؛ <code>fetch_oi</code> ستون‌های <code>OpenInterest, ContractSize, BeginDate, EndDate</code> .	؛ مثال <code>underlying/provider errors</code> ؛ فصل اختیار <code>chain</code> .
<code>black_scholes_price</code>	فقط European: scalar math params.	<code>float</code> premium.	نامعتبر <code>bounds/type/style</code> ؛ مثال <code>ValueError</code> ؛ <code>deterministic</code> .
<code>black_scholes_greeks</code>	ورودی‌های <code>spot/strike/time_to_expiry/rate/volatility</code> عددی؛ <code>option_type∈{call,put}</code> ؛ <code>dividend_yield</code> نرخ پیوسته؛ <code>exercise_style='european'</code> تنها سبک پشتیبانی‌شده.	<code>dict[Delta, Gamma, Vega, Vega1Pct, ThetaPerYear, ThetaPerDay, Rho, Rho100bp, Status]</code> ؛ یا <code>Status='ok'</code> یا <code>undefined_at_expiry_or_zero_volatility</code> .	نامعتبر <code>constraint</code> ؛ مثال <code>ValueError</code> ؛ <code>Greeks</code> .
<code>option_price_bounds</code>	بدون <code>volatility</code> ؛ <code>no-arbitrage bound</code> .	<code>tuple (lower: float, upper: float)</code> .	مثال <code>ValueError</code> ؛ <code>deterministic</code> با خروجی <code>(4.87705755, 100.0)</code> .
<code>implied_volatility</code>	<code>premium + bracket/tolerance/iterations: tolerance>0</code> ، عدد صحیح مثبت <code>max_iterations</code> و <code>0<=lower<upper</code> .	با <code>dict ImpliedVolatility, Status, Iterations</code> ؛ <code>status</code> های <code>ok/missing/expiry/out_of_bounds/no_bracket/non_converged</code> و کلیدهای تشخیصی اختیاری.	و <code>bounds</code> ناموجود/خارج <code>premium</code> هستند، نه <code>status</code> عدم همگرایی؛ فقط <code>exception</code> ؛ نامعتبر <code>constraint/style/bracket</code> ؛ مثال <code>ValueError</code> ؛ <code>IV</code> .

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_option_mark</code> <code>et</code>	<code>exchange: int=0 ; underlying</code> <code>filter: max_requests=1 bulk.</code>	<code>DataFrame[InsCode, PairID</code> <code>, PairSequence, ISIN, Symbol</code> <code>, Name, OptionType, Underlyi</code> <code>ngInsCode, UnderlyingSymbo</code> <code>l, UnderlyingName, Contract</code> <code>Size, Strike, BeginDate, End</code> <code>Date, DaysToExpiry, Last, Cl</code> <code>ose, Yesterday, Volume, Valu</code> <code>e, TradeCount, NotionalValu</code> <code>e, OpenInterest, YesterdayO</code> <code>penInterest, BidPrice, AskP</code> <code>rice, BidVolume, AskVolume,</code> <code>UnderlyingLast, Underlying</code> <code>Close, Price, PriceSource, A</code> <code>sOf, AsOfSource, SnapshotFr</code> <code>eshnessKnown, PriceFreshne</code> <code>ssKnown, Stale, NoTrade, Ana</code> <code>lyticsEligible, AnalyticsE</code> <code>ligibilityReason, Metadata</code> <code>Conflict, Source] ; attrs</code> <code>snapshot provenance.</code>	<code>exchange/budget/filter</code> <code>InvalidParameterError ;</code> <code>provider typed</code> مثال: <code>snapshot.</code>
<code>analyze_option_</code> <code>chain</code>	می‌کند؛ <code>fetch</code> <code>options=None</code> <code>spot/rate/curve overrides:</code> <code>allow_unverified_freshnes</code> <code>s</code> <code>explicit risk switch.</code>	<code>input columns +</code> <code>TimeToExpiry, Spot, RiskFr</code> <code>eeRate, DividendYield, Impl</code> <code>iedVolatility* , Greeks per</code> <code>unit/contract.</code> <code>spread/depth/liquidity.</code> <code>ParityResidual, ParitySta</code> <code>tus, ImpliedForward, Analyt</code> <code>icsReliability ; attrs</code> <code>assumptions/warnings.</code>	<code>input type TypeError ;</code> <code>math/freshness/rate/column</code> <code>problems ValueError ;</code> <code>provider errors if fetch</code> مثال: <code>حرفه‌ای.</code>
<code>option_put_call</code> <code>_ratios</code>	از یک <code>options: DataFrame</code> دقیقاً یکی <code>group_by</code> <code>Atmیک</code> ؛ <code>AsOf</code> از <code>market, underlying, expiry, u</code> <code>nderlying_expiry .</code>	<code>DataFrame</code> با <code>call/put</code> <code>volume/value/OI.</code> <code>PCRVOLUME, PCRValue, PCROp</code> <code>enInterest</code> و <code>status</code> دقیق و <code>PCRVOLUMEStatus, PCRValue</code> <code>Status, PCROpenInterestSta</code> <code>tus ; attrs coverage/source</code> ورودی	ستون/ <code>AsOf</code> یا <code>group</code> یا <code>قرارداد تکراری</code> نامعتبر <code>ValueError</code> و نوع غیر <code>TypeError</code> <code>DataFrame</code> مثال <code>PCR.</code>

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_option_history</code>	قرارداد دقیق؛ + <code>server history</code> <code>include_today</code> ; فقط <code>snapshot_path</code> join <code>snapshot</code> های <code>opt-in</code>: <code>budget</code>.	<code>DataFrame[Timestamp, InsCode, Symbol, Open, High, Low, Close, Last, Volume, Value, TradeCount, OpenInterest, BidPrice, AskPrice, BidVolume, AskVolume, UnderlyingLast, UnderlyingClose, ContractSize, Strike, EndDate, Price, PriceSource, Source, AsOf, Stale, NoTrade, AnalyticsEligible]</code> ; <code>attrs failures/no-lookahead</code>.	<code>selector/date/budget/provider/snapshot</code> مثال تاریخچه <code>schema errors</code>؛ اختیار.
<code>save_option_snapshot</code>	<code>fetch</code>: lock یک <code>options=None</code> مثبت stale lock نامنفی و <code>timeout</code>.	شده با <code>dedupely</code> ترکیب <code>DataFrame</code> <code>OPTION_COLUMNS</code> ; <code>attrs schema_version, source, path</code> و <code>lock</code> با <code>JSON versioned</code> فایل . اتمیک نوشته می‌شود <code>replace</code>.	گم‌شده <code>parent</code> <code>FileNotFoundError</code> , <code>DataFrame</code> غیر <code>options</code> مقدار نامعتبر , <code>TypeError</code> قفل , <code>ValueError</code> اگر <code>provider</code> <code>TimeoutError</code> ; <code>snapshot</code> ; مثال <code>fetch</code>.
<code>load_option_snapshots</code>	<code>JSON versioned</code>. <code>path</code>	با <code>DataFrame</code> ؛ فایل <code>OPTION_COLUMNS</code> و <code>attrs</code> تهی با <code>frame</code> گم‌شده خطا نیست و <code>attrs['status']='missing'</code> می‌دهد .	خراب <code>JSON</code> و <code>version</code> <code>JSONDecodeError</code> مثال ; <code>ValueError</code> ناسازگار <code>history</code>.
<code>list_etfs</code>	فقط <code>progress</code> .	<code>DataFrame[InsCode, ISIN, Symbol, Name, Last, Close, Yesterday, Volume, Value, TradeCount, Low, High, NAV, NAV_Discout, Change, ChangePct, MarketCode]</code> .	مثال <code>provider typed/empty</code> ; <code>list_etfs().query("NAV_Discout < -1")</code> .
<code>list_bonds</code>	فقط <code>progress</code> .	<code>DataFrame[InsCode, ISIN, Symbol, Name, BondType, Ticker, MaturityJalali, MaturityGregorian, DaysToMaturity, Last, Close, Yesterday, Volume, Value, TradeCount, Change, ChangePct]</code> .	نامعلوم <code>maturity</code>: <code>provider</code>: <code>parse</code> ؛. مثال فصل ابزارها <code>nullable</code>
<code>list_funds</code>	از <code>fund_type: str list None</code> <code>categories settings</code>: <code>listed_only=False</code> ; <code>listed_only</code> قابل ترکیب نیست <code>filter type</code> با	<code>registry rich funds</code> (<code>NAV/returns/composition/manager</code>) ؛ بدون تضمین <code>InsCode</code> با دقیقاً <code>listed_only=True</code> <code>list_listed_funds</code> . <code>schema</code> <code>attrs no_fuzzy_join=True</code> .	نامعتبر <code>type/category/combo</code> <code>ValueError</code> ; <code>provider errors</code>: مثال <code>funds</code>.
<code>list_listed_funds</code>	فقط <code>progress</code> ; یک <code>MarketWatch</code> <code>bulk</code>.	<code>LISTED_FUND_COLUMNS</code> <code>attrs</code> ; <code>no_fuzzy_join=True, registry_joined=False</code> .	مثال فصل <code>connection/parsing</code> <code>صندوق</code>.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>list_indices</code>	فقط <code>progress</code> .	<code>DataFrame[Name, InsCode, Value, High, Low, Change, ChangePct]</code> حرکت <code>Change</code> از <code>gePct</code> 1.2.0 درصد <code>ChangePct</code> واحد شاخص و است.	مسیر legacy در خطای provider frame تهی/پیام؛ برای صنعت <code>list_industry_indices</code> پیشنهاد می‌شود.
<code>get_index_companies</code>	فارسی یا <code>index_name: str</code> InsCode.	<code>DataFrame[Symbol, Name, InsCode, Close, Yesterday, Last]</code>	در مسیر provider/گم‌شده index تهی/پیام؛ مثال فصل legacy frame شاخص.

قرارداد alias‌ها و wrapperهای backward-compatible

قدیمی و تفاوت رفتاری مخفی نماند. تمام پارامترها signature های این جدول مدخل مستقل دارند تا alias است. override را دارند؛ فقط تفاوت صریح جدول target تابع type/default/constraint

```

stock(symbol='', start=None, end=None, limit=0, raw=False, auto_adjust=True, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, adjust_volume=False,
return_type=None, ascending=True, save_path=None, include_today=False, *, ins_code=None,
asset_type='auto', **kwargs)
stock_RI(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, *, ins_code=None, asset_type='auto', **kwargs)
stock_RL(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, *, ins_code=None, asset_type='auto', **kwargs)
stock_capital_increase(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stock_intraday(symbol='شتران', interval='1min', start=None, end=None, progress=True, **kwargs)
stockdetail(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stock_information(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stock_statistics(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stock_introduction(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stocklist(bourse=True, farabourse=True, payeh=True, haghe_taqadom=False, sandogh=False, bonds=False,
options=False, mortgage=False, commodity=False, energy=False, payeh_color=None, output='dataframe',
progress=True, **kwargs)
shareholders(symbol='', date=None, include_id=False, *, ins_code=None, asset_type='auto', **kwargs)
currency_coin(name='', start=None, end=None, limit=0, output_type='standard', date_format='jalali',
progress=True, save_to_file=False, dropna=True, return_type=None, ascending=True, save_path=None,
**kwargs)
market_watch()
market_client_type()
market_data()

```

alias/wrapper	و تفاوت target	خروجی/schema/attrs	خطا و مثال
<code>stock</code>	همان <code>get_history</code> target signature و implementation history.	همان DataFrame/attrs.	همان خطاها؛ <code>att.stock("فملی", .limit=10)</code>

alias/wrapper	و تفاوت target	خروجی/schema/attrs	خطا و مثال
stock_RI	target <code>get_client_type</code> .	همان DataFrame/attrs.	همان؛ <code>att.stock_RI("فملى", .limit=10)</code>
stock_RL	که به wrapper compatibility identity می‌کند؛ delegate <code>stock_RI</code> signature نیست ولی alias برابر.	همان DataFrame/attrs.	همان؛ برای کد جدید <code>.get_client_type</code>
stock_capital_increase	target canonical <code>get_capital_increase</code> .	همان frame/None قدیمی.	همان.
stock_intraday	target <code>get_intraday</code> ; signature برابر.	همان tick/candle frame.	همان.
stockdetail	target <code>get_detail</code> .	همان key/value frame یا <code>None</code> .	همان.
stock_information	target <code>get_info</code> .	همان key/value frame یا <code>None</code> .	همان.
stock_statistics	target <code>get_stats</code> .	همان key/value frame یا <code>None</code> .	همان.
stock_introduction	target <code>get_introduction</code> ; wrapper legacy unsupported.	هیچ خروجی موفق.	همیشه <code>UnsupportedDataSourceError</code> <code>ror</code> پیش از I/O.
stocklist	همان؛ target <code>get_symbols</code> ; signature و alias های kwargs.	همان فهرست.	همان.
shareholders	target <code>get_shareholders</code> .	همان frame/None.	همان.
currency_coin	target <code>get_currency</code> .	همان frame/MultiIndex و source .TGJU	همان.
market_watch	تابع صفرآرگومان canonical قدیمی target snapshot؛ مفهومی <code>.get_market_snapshot</code>	<code>dict[stocks,order_book,market_time,...]</code> ; ستون‌های legacy <code>Yesterday/BaseVolume/Change</code> حفظ شده و ستون‌های corrected additive هستند.	<code>ConnectionError/DataParingError</code> ; <code>snapshot=att.market_watch()</code> .
market_client_type	تابع صفرآرگومان bulk؛ target مفهومی <code>.get_market_client_type</code>	<code>CLIENT_COLUMNS</code> DataFrame.	typed provider errors.
market_data	صفرآرگومان به wrapper deprecated <code>market_watch</code> ; identity alias نیست.	همان dict.	همان؛ برای کد جدید <code>get_market_snapshot/get_live_market</code>

دو alias identity غیر legacy نیز قبلاً مدخل دارند: `get_orderbook_history is get_order_book_history` و `get_treasury_yields_history is get_treasury_yield_history`. تفاوت signature یا خروجی ندارند.

ها نیز دقیقاً چنین است signature constructor exception

```

AlgotikTSEError(*args)
AmbiguousSymbolError(*args)
ConnectionError(*args)
DataParsingError(*args)
InvalidParameterError(*args)
StockNotFoundError(*args)
UnsupportedDataSourceError(*args)

```

الگوهای کاربردی

فیلتر قدرت خریدار حقیقی با نقدشوندگی

```

live = att.get_live_market()
screen = live.loc[
    (live["IndividualPower"] > 1.5)
    & (live["Value"] > 50_000_000_000)
    & live["is_realtime_fresh"].fillna(False)
].sort_values("EstimatedNetIndividualFlow", ascending=False)

print(screen[[
    "Symbol", "Last", "ChangePct", "IndividualPower",
    "EstimatedNetIndividualFlow", "SpreadBps", "L5Imbalance",
]])

```

look-ahead بدون backtest

```

db = "research.sqlite"
att.save_market_snapshot(db)

fund = att.get_market_fundamentals_history(db, symbols="فملی")
assert fund.attrs["no_lookahead"] is True
assert fund.attrs["current_eps_used"] is False

curves = att.get_yield_curve_history(
    start="1403-01-01", end="1403-03-31", max_requests=100, progress=False
)
safe_curves = curves.loc[curves["CurveNoLookahead"].fillna(False)]

```

مانیتور بازار و archive مستقل

```
db = "monitor.sqlite"

for event in att.watch_market(
    interval=3,
    max_updates=20,
    record_to=db,
    notifications=("messages", "state"),
):
    if event.kind in {"initial", "delta"}:
        print(event.sequence, len(event.changed_inscodes))

# مستقل فعال شود helper باید روی archive_to
att.get_market_messages(flow=0, top=50, archive_to=db)
messages = att.get_market_messages_history(db, flow=0)
```

منابع داده

منبع	استفاده
های subdomain و tsetmc.com (TSETMC رسمی)	قیمت، client type، market watch، سفارش، trades، پیام، وضعیت، ابزار، صندوق و اطلاعات بازار
فراپورس ایران (ifb.ir/ytm.aspx)	جدول مرجع YTM برای مقایسه/دسته‌بندی اوراق؛ منبع مجاز و با provenance جدا
TGJU (api.tgju.org)	فقط API legacy ارز و سکه

کدال منبع این پکیج نیست؛ حتی endpointهای proxy شده آن زیر host دیگر در boundary شبکه رد می‌شوند.

تست و مشارکت

تست پیش‌فرض کاملاً آفلاین است:

```
python -m pytest -m "not online"
```

یا:

```
make test
```

است: opt-in آنلاین محدود و smoke

```
python -m pytest -m online --timeout=30
```

تست آنلاین به وضعیت بازار/provider وابسته است و جایگزین تست deterministic آنلاین نیست. پیش از PR:

```
python -m pytest tests/test_release_offline.py -q
python -m pytest -m "not online" -q
```

برای مشارکت، issue یا pull request در [GitHub](#) باز کنید. انتشار PyPI فقط با مسیر دستی و تأیید صریح انجام می‌شود؛ target عادی [release](#) صرفاً artifact محلی می‌سازد و upload نمی‌کند.

مجوز و ارتباط

این پروژه تحت [GNU General Public License v3](#) منتشر می‌شود.

- وبسایت: [algotik.com](#)
- تلگرام: [t.me/algotik](#)
- نویسنده: [alipour@algotik.ir](#) — Mohsen Alipour